# E-COMMERCE MANAGEMENT SYSTEM

**A PROJECT REPORT**

*Submitted by*

**Kishore.v**

240701270

*in partial fulfilment of the End Semester Examination for the course*

**CB23F35 –**

**LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR**

*in*

**BACHELOR OF ENGINEERING**

**COMPUTER SCIENCE AND ENGINEERING**




**RAJALAKSHMI ENGINEERING COLLEGE**

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

**NOVEMBER 2026**

**RAJALAKSHMI ENGINEERING COLLEGE**

(An Autonomous Institution, Affiliated to Anna University, Chennai)

# BONAFIDE CERTIFICATE

Certified that this project report titled “E-COMMERCE MANAGEMENT SYSTEM” is the bonafide work of Kishore.v 240701270 , who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

| **SIGNATURE** | **SIGNATURE** |
|---|---|
| Dr. J. MANORANJANI<br>HEAD OF THE DEPARTMENT<br>Department of Computer Science and Engineering<br>Rajalakshmi Engineering College<br>Thandalam – 602 105 | Ms. ASWANA LAL<br>COURSE FACULTY<br>Assistant Professor<br>Department of Computer Science and Engineering<br>Rajalakshmi Engineering College |

Submitted for the End Semester Examination (Individual Project) held on 10 November 2026.

| **INTERNAL EXAMINER** | **EXTERNAL EXAMINER** |
|---|---|

# DECLARATION

I, Kishore.v 240701270 , hereby declare that the project report titled “E-COMMERCE MANAGEMENT SYSTEM”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date: 10 November 2026

**SIGNATURE OF THE STUDENT**
Kishore.v
240701270

# ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. Abhay Meganathan S for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. J. Manoranjani, Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Ms. Aswana Lal, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation and the Zoho Creator Education Programme for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

Kishore.v

# ABSTRACT

Small and medium online retailers frequently manage products, customers, orders and courier shipments through spreadsheets, messaging applications and handwritten notes. Product stock is checked by eye, duplicate customer entries are common, order status is communicated by phone, and delayed deliveries are noticed only after a customer complains. These gaps cost staff time, produce inconsistent records and give the owner no real-time view of sales or pending deliveries.

This project presents the E-Commerce Management System, a business application built on the Zoho Creator low-code platform to replace that manual process. A low-code approach was chosen because the data model is form-centric, the automation is rule-based, and the owner must be able to modify the application without a development team; Creator also provides hosting, security, reporting and mobile access out of the box. The application consists of eight forms (product categories, products, customer registrations, customer login, order booking, orders, an order-placement form and deliveries), eleven reports including list, summary and Kanban views, and an administrator dashboard page with KPI tiles, charts and a ranked board. Business rules are implemented in Deluge: sixteen scripts run as fourteen form workflows, one scheduled workflow, one custom action and one standalone function. They block duplicate customers and categories, validate product and delivery data, calculate the discounted unit price, verify stock before booking, create delivery records automatically, e-mail customers when order status changes, send a push notification on delivery, and alert the administrator when a delivery is overdue. Access is organised through Read, Write, Developer, Administrator and Customer (portal) profiles and a CEO role.

The application was verified through unit, workflow and role-based test cases mapped to the functional requirements, and a code review that produced a defect log. The report documents the design, implementation, testing, deployment and governance of the system and critically discusses the limitations of the low-code approach encountered during development. Five manual activities per order — stock checking, duplicate-customer checking, creating the delivery record, notifying the customer and following up overdue parcels — are now performed by the system, so the administrator enters only the order itself, and order counts, revenue and status distribution are available instantly on the dashboard.

**Keywords:** Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Role-Based Access Control; E-Commerce

# PROJECT AT A GLANCE

| | |
|---|---|
| **Project Title** | E-Commerce Management System |
| **Application Domain** | Retail / E-Commerce (order, product and delivery management) |
| **Student Name & Register No.** | Kishore.v – 240701270 |
| **Zoho Creator Edition** | Zoho Creator – Education Edition (student account) |
| **Application Name (Link Name)** | E Commerce Management System (E_Commerce_Management_System) |
| **Published / Live URL** | https://creatorapp.zoho.in/dhanush.v.2024.cse_rajala366/e-commerce-management-system |
| **Demo Credentials (per role)** | Administrator: application owner account (dhanush.v.2024.cse_rajala366); password shared with the examiner separately.<br>Customer: any Active customer e-mail registered in Customer Registrations. |
| **Number of Forms / Subforms** | 8 / 0 |
| **Number of Reports (by type)** | List: 8 \| Kanban: 2 \| Summary: 1 \| Calendar: 0 \| Pivot/Chart reports: 0 (charts are on the dashboard page) |
| **Number of Pages / Dashboards** | 1 (Admin Dashboard) |
| **Number of Workflows** | Form: 14 \| Schedule: 1 \| Approval: 0 \| Blueprint: 0 \| Batch: 0 \| Custom action (function workflow): 1 |
| **Deluge Functions / Custom Actions** | 1 standalone function (add_sample_catalog) / 1 custom action (Mark as Delivered) |
| **Integrations** | Zoho Mail (sendmail task); Creator push notifications. No external REST integration. |
| **Roles & Profiles Defined** | Profiles: Read, Write, Developer, Administrator, Customer (Portal). Role: CEO. |
| **Environments Used** | Development (application built, tested and demonstrated here) |
| **Date of Production Deployment** | 4 November 2026 (demonstration from the Development environment) |

# TABLE OF CONTENTS

**CHAPTER 1 INTRODUCTION** ..... 1
1.1 Background ..... 1
1.2 Problem Statement ..... 1
1.3 Objectives ..... 1
1.4 Scope of the Project ..... 2
1.5 Justification for a Low-Code Approach ..... 2
1.6 Organisation of the Report ..... 2
**CHAPTER 2 LITERATURE SURVEY AND SYSTEM STUDY** ..... 3
2.1 Study of the Existing System ..... 3
2.2 Limitations of the Existing System ..... 3
2.3 Low-Code versus Traditional Development ..... 4
2.4 Literature Survey ..... 4
2.5 Emerging Trends in Low-Code Development ..... 5
2.6 Proposed System ..... 5
**CHAPTER 3 REQUIREMENT ANALYSIS** ..... 6
3.1 Stakeholders and User Roles ..... 6
3.2 Functional Requirements ..... 6
3.3 Non-Functional Requirements ..... 7
3.4 Hardware and Software Requirements ..... 7
3.5 Use Case Model ..... 8
**CHAPTER 4 SYSTEM DESIGN** ..... 10
4.1 System Architecture ..... 10
4.2 Data Model ..... 10
4.3 Form Design ..... 11
4.4 Process and Workflow Design ..... 15
4.5 User Interface and Navigation Design ..... 15
4.6 Access Control Design ..... 16
**CHAPTER 5 IMPLEMENTATION** ..... 17
5.1 Application Setup ..... 17
5.2 Forms and Data Capture ..... 17
5.3 Reports ..... 18
5.4 Pages and Dashboards ..... 19
5.5 Deluge Scripting ..... 20
5.6 Workflows and Automation ..... 22
5.7 Integrations ..... 23
5.8 User Management and Role-Based Access ..... 23
5.9 AI and Analytics Features ..... 24
5.10 Pro-Code Extension ..... 24
5.11 Mobile and Portal Access ..... 24
**CHAPTER 6 TESTING AND DEBUGGING** ..... 26
6.1 Testing Strategy ..... 26
6.2 Test Cases ..... 26

6.3 Debugging Deluge Scripts....28
6.4 Defect Log....28
6.5 User Acceptance Testing....29
**CHAPTER 7 DEPLOYMENT, SECURITY AND GOVERNANCE**....30
7.1 Environments and Deployment Strategy....30
7.2 Performance Optimisation....30
7.3 Security Features....31
7.4 Governance and Administration....31
**CHAPTER 8 RESULTS AND DISCUSSION**....33
8.1 Application Output....33
8.2 Achievement of Objectives....34
8.3 Comparison with the Existing System....34
8.4 Limitations of the Low-Code Approach....34
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum....35
**CHAPTER 9 CONCLUSION AND FUTURE ENHANCEMENTS**....36
9.1 Conclusion....36
9.2 Future Enhancements....36
9.3 Learning Outcomes and Career Relevance....36
**REFERENCES**....37
**APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST**....38
**APPENDIX B – COURSE OUTCOME MAPPING**....40
**APPENDIX C – COMPLETE DELUGE SOURCE CODE**....41
**APPENDIX D – USER MANUAL**....50
D.1 Accessing the Application....50
D.2 Instructions for the Administrator....50
D.3 Instructions for the Customer (Portal User)....50
**APPENDIX E – EVALUATION RUBRIC**....51
**APPENDIX F – VIVA-VOCE RECORD SHEET**....52

# LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application....4
Table 2.2: Summary of Literature Survey....4
Table 3.1: Stakeholders and User Roles....6
Table 3.2: Functional Requirements....6
Table 3.3: Non-Functional Requirements....7
Table 3.4: Hardware and Software Requirements....7
Table 3.5: Use Case Description – Place an Order and Track Delivery....8
Table 3.6: Use Case Description – Register a Customer....9
Table 4.1: Entities and Relationships....11
Table 4.2: Field Specification – Customer Registrations Form....11
Table 4.3: Field Specification – Products Form....12
Table 4.4: Field Specification – Orders Form....12
Table 4.5: Field Specification – Deliveries Form....13
Table 4.6: Field Specification – Order Booking Form....13
Table 4.7: Field Specification – Product Categories Form....14
Table 4.8: Field Specification – Customer Login Form....14
Table 4.9: Field Specification – Orders (Order Placed, orders1) Form....14
Table 4.10: Navigation Structure....15
Table 4.11: Role–Permission Matrix (A = Add, E = Edit, V = View, D = Delete, — = No access)....16
Table 5.1: Reports Implemented....18
Table 5.2: Deluge Scripts Implemented....20
Table 5.3: Workflows Implemented....22
Table 5.4: Integrations Implemented....23
Table 6.1: Test Cases....26
Table 6.2: Defect Log....28
Table 6.3: User Acceptance Feedback....29
Table 7.1: Deployment Checklist....30
Table 7.2: Performance Optimisation Results....31
Table 7.3: Security Controls Implemented....31
Table 8.1: Mapping of Objectives to Outcomes....34
Table 8.2: Existing System versus Proposed Application....34
Table A.1: Feature Coverage Checklist....38
Table B.1: Mapping of Course Outcomes to Report Chapters....40
Table E.1: Evaluation Rubric for the Individual Project....51
Table E.2: Performance Level Descriptors....51
Table F.1: Viva-Voce Questions and Responses....52
Table F.2: Marks Summary....52

# LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System....3
Figure 3.1: Use Case Diagram of the Application....8
Figure 4.1: System Architecture of the Application....10
Figure 4.2: Entity–Relationship Diagram....10
Figure 4.3: Workflow / Process Flow for Order Placement, Shipment and Delivery....15
Figure 5.1: Application Home and Navigation Sections....17
Figure 5.2: Products Form in the Form Builder....18
Figure 5.3: Orders Form in Live Mode....18
Figure 5.4: Delivery Statuses (Kanban) Report....19
Figure 5.5: Admin Dashboard Page....19
Figure 5.6: Workflow Configuration....23
Figure 5.7: Roles and Permission Profiles Configuration....24
Figure 5.8: Analytics on the Admin Dashboard....24
Figure 5.9: Application on Mobile / Customer Portal....25
Figure 6.1: Deluge Debug Output / Log....28
Figure 7.1: Environments / Publish to Production....30
Figure 7.2: Audit Trail / Administration Settings....32
Figure 8.1: Scenario Step 1 – Customer Registration....33
Figure 8.2: Scenario Step 2 – Order Placed and Delivery Created....33
Figure 8.3: Scenario Step 3 – Status E-mail and Dashboard....33

# LIST OF ABBREVIATIONS

| Abbreviation | Expansion |
|---|---|
| API | Application Programming Interface |
| CRUD | Create, Read, Update, Delete |
| CSS | Cascading Style Sheets |
| ER | Entity–Relationship |
| FR | Functional Requirement |
| INR | Indian Rupee |
| KPI | Key Performance Indicator |
| LCAP | Low-Code Application Platform |
| NFR | Non-Functional Requirement |
| PII | Personally Identifiable Information |
| RBAC | Role-Based Access Control |
| REST | Representational State Transfer |
| SDLC | Software Development Life Cycle |
| UAT | User Acceptance Testing |
| UI | User Interface |
| Zia | Zoho’s AI assistant |

# CHAPTER 1
# INTRODUCTION

## 1.1 Background

Online retail has grown rapidly in India, and a large share of sellers are small businesses that begin with a handful of products, a phone, and a spreadsheet. Such a business must repeatedly perform the same set of activities: maintain a product catalogue organised into categories, record customers, accept orders, hand the parcel to a courier, and keep the customer informed until the parcel is delivered. The people involved are the shop owner or administrator, who maintains products and monitors orders, and customers, who place orders and expect timely updates.

The organisation considered in this project is a small online retailer selling items across categories such as Electronics, Clothing and Home & Garden, and shipping through couriers such as India Post, Blue Dart, Delhivery, DTDC and Professional Couriers. Today each activity is handled in a separate place — a spreadsheet for products, a notebook for orders, a courier portal for shipments — and the only link between them is the owner's memory.

## 1.2 Problem Statement

Retailers of this scale lack a single, consistent system that connects products, customers, orders and deliveries, enforces basic business rules, and notifies customers automatically. The result is stock errors, duplicate records, delayed delivery follow-up and no real-time visibility of sales performance.

- Product stock is verified manually, so orders are accepted for items that are out of stock.
- The same customer is entered several times with different spellings, and contact details diverge.
- Order status changes are communicated by phone or message, one customer at a time.
- Courier details and tracking numbers are kept apart from the order, and overdue deliveries are noticed late.
- Totals such as total orders and revenue are computed by hand at month end.

### 1.3 Objectives

- To design a normalised data model and forms for categories, products, customers, orders and deliveries.
- To automate duplicate checking, stock and price validation, delivery creation and status notification using Deluge scripting and workflows.
- To provide an administrator dashboard and reports (list, summary and Kanban) for orders, deliveries and products.
- To use Zoho Creator's built-in e-mail and push-notification capabilities to keep customers and administrators informed.
- To control access through profiles and roles, and to deploy and govern the application on Zoho Creator.

### 1.4 Scope of the Project

In scope: the product catalogue (categories and products), customer registration and a login-validation form, order booking with automatic unit-price calculation and stock validation, order management linked to a registered customer, delivery tracking with courier and status, e-mail and push notifications, a scheduled overdue-delivery alert, an administrator dashboard and reports. The application is delivered as a web application and is also available on the Zoho Creator mobile and tablet apps through the same component set; the Customer profile is of type Customer Portal.

Out of scope: online payment, invoicing, multi-item (subform) order lines, integration with courier tracking APIs, tax computation and inventory purchasing.

### 1.5 Justification for a Low-Code Approach

Low-code application platforms let developers and trained business users build applications through visual builders, supplemented with a small amount of scripting, and are valued for development speed and lower maintenance effort [1], [5]. The proposed system is a good fit: its data model is form-centric, its automation is rule-based (validate, create a related record, send a message), and its reports are standard list, summary and Kanban views. Zoho Creator supplies hosting, authentication, responsive web and mobile access, role-based security and a scripting language (Deluge) in one platform [2], [3], so no infrastructure, front-end framework or API layer had to be built. The owner, who is not a professional programmer, can later add a field or change a status list directly in the builder.

### 1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

# CHAPTER 2
# LITERATURE SURVEY AND SYSTEM STUDY

## 2.1 Study of the Existing System

In the existing process the customer contacts the seller by phone, message or in person. The seller notes the order in a spreadsheet or notebook, checks stock by looking at the shelf or an inventory sheet, books a courier through the courier's own portal or counter, and informs the customer of progress by call or message. At month end the owner adds up order values by hand.

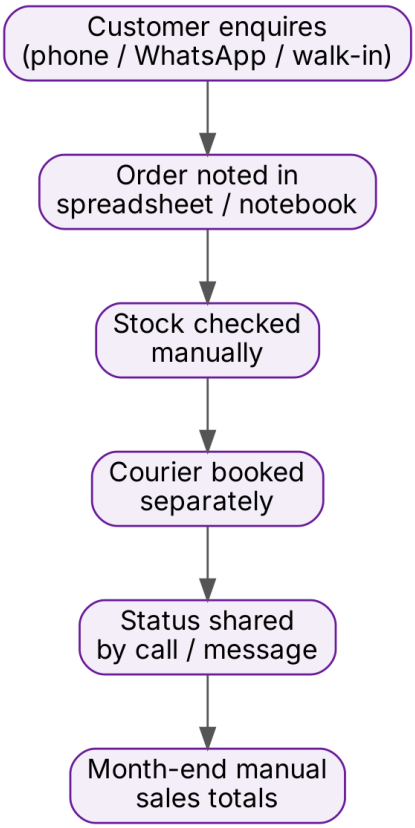



Figure 2.1: As-Is Process Flow of the Existing System

## 2.2 Limitations of the Existing System

- No single source of truth: products, customers, orders and shipments live in different tools and are not linked.
- No validation: duplicate customers, negative prices, out-of-stock orders and impossible delivery dates are accepted.
- Notifications are manual, so customers are updated late or not at all.
- No reminder for overdue deliveries; follow-up depends on memory.
- No dashboard; revenue and order-status counts need manual calculation.

- No access control; anyone with the spreadsheet can change any record.

### 2.3 Low-Code versus Traditional Development

Table 2.1 compares the two approaches for this application. Effort figures are the author's estimates.

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

| Criterion | Traditional (Pro-Code) Development | Low-Code on Zoho Creator |
|---|---|---|
| Development time | Approx. 6–8 weeks (schema, back-end API, web UI, authentication, e-mail service) | Approx. 1–2 weeks (forms, reports, page and Deluge scripts) |
| Skills required | Back-end and front-end programming, database design, DevOps | Data modelling, Creator builders, basic Deluge |
| Hosting & infrastructure | Server or cloud setup, database, SSL, backups to be arranged | Hosted by Zoho with SSL and backups included |
| Mobile application | Separate responsive build or native app | Available through the Zoho Creator mobile / tablet app |
| Security & access control | Custom authentication and authorisation code | Built-in profiles, roles and portal |
| Maintenance & change | Code change, test and redeploy | Edit in the builder and publish |
| Customisation limits | Practically none | Bounded by platform features and Deluge limits |
| Cost | Developer time plus hosting | Per-user licence (education access for this project) |

### 2.4 Literature Survey

Table 2.2 summarises five sources reviewed for this project.

Table 2.2: Summary of Literature Survey

| S.No. | Author(s) / Source & Year | Title / Product | Key Findings | Relevance to this Project |
|---|---|---|---|---|
| 1 | Zoho Corporation [2] | Zoho Creator Help Documentation | Describes forms, reports, pages, workflows, roles and deployment in Creator. | Primary reference for platform features used. |
| 2 | Zoho Corporation [3] | Deluge Script Reference Guide | Defines Deluge syntax, data access, task statements and form events. | Basis of all scripts in Chapter 5. |

| S.No. | Author(s) / Source & Year | Title / Product | Key Findings | Relevance to this Project |
|---|---|---|---|---|
| 3 | Sahay et al. [6], 2020 | Supporting the understanding and comparison of low-code development platforms (SEAA) | Proposes a framework for comparing LCDP features and capabilities. | Supports the platform justification in Section 1.5. |
| 4 | Luo et al. [7], 2021 | Characteristics and challenges of low-code development: the practitioners' perspective (ESEM) | Reports practitioner views: speed benefits, but limits in customisation and learning curve. | Informs limitations in Section 8.4. |
| 5 | Bock and Frank [8], 2021 | Low-Code Platform (BISE) | Discusses the nature and organisational implications of low-code platforms. | Context for citizen development in Section 2.5. |
| 6 | Shopify, WooCommerce (products) | Hosted and plug-in e-commerce platforms | Offer full storefronts, payments and catalogues as products. | Reference for the domain; this project targets internal back-office management rather than a storefront. |

### 2.5 Emerging Trends in Low-Code Development

Current trends include AI-assisted application building, citizen development, hyper-automation and composable applications [1], [5], [8]. Platform vendors now offer AI assistants that generate forms or scripts from a description, and Zoho Creator exposes Zia-based features for fields and analysis. These trends influenced the design in two ways: the application was kept as a set of small, single-purpose forms and workflows that a citizen developer can understand and change, and each automation was written as a separate named workflow rather than one large script, which suits composable reuse. Generated Deluge was treated as a draft to be reviewed and tested, as the defect log in Chapter 6 shows.

### 2.6 Proposed System

The proposed system is a Zoho Creator application that holds products, customers, orders and deliveries in linked forms. Duplicate and invalid data are rejected at submission time (limitations 2 and 6). Delivery records are created automatically when an order is placed or shipped, and customers are e-mailed when order status changes (limitation 3). A scheduled workflow alerts the administrator one day after the expected delivery date if the parcel is not delivered (limitation 4). An administrator dashboard and reports show order counts, revenue, status distribution and delivery distribution without manual calculation (limitation 5), and profiles and roles restrict who can see

and change data (limitation 6). Linked forms provide the single source of truth (limitation 1).

# CHAPTER 3
# REQUIREMENT ANALYSIS

## 3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

| Role | Description | Key Responsibilities in the Application |
|---|---|---|
| Administrator | Owner or manager of the shop (Administrator profile) | Manage products, categories, customers, orders and deliveries; view dashboard and reports; receive overdue alerts |
| Developer | Person maintaining the application (Developer profile) | Modify forms, workflows and Deluge scripts |
| Staff (Read / Write) | Shop staff using the Read or Write profile | View or add and edit records according to the profile |
| Customer (Portal user) | Registered shopper (Customer portal profile: add and view only) | Register, place an order, receive status e-mails and delivery notifications |

## 3.2 Functional Requirements

Table 3.2: Functional Requirements

| FR ID | Requirement | Role(s) | Priority |
|---|---|---|---|
| FR-01 | The system shall allow registration of a customer with name, e-mail, phone, address, status and an auto-generated customer code; registration date and status shall default to today and Active. | Admin, Customer | High |
| FR-02 | The system shall reject a customer whose e-mail or phone number is already registered. | Admin | High |
| FR-03 | The system shall maintain product categories and reject a category whose name already exists. | Admin | Medium |
| FR-04 | The system shall maintain products with price, discount, stock and category, reject negative price, negative stock and discount above 100, and set the status to Out of Stock when stock is zero or less. | Admin | High |
| FR-05 | On order booking the system shall compute the unit price after discount from the selected product, and block the booking if the product is not found, out of stock or the quantity exceeds stock. | Admin, Staff | High |
| FR-06 | The system shall record an order linked to a registered customer with date, delivery address, status and total amount. | Admin, Customer | High |
| FR-07 | When an order is created the system shall automatically create a delivery record with status Pending. | System | High |
| FR-08 | When an order is edited and its status is Shipped, the system shall create a delivery record with status Shipped, the shipped date and a default courier. | System | Medium |
| FR-09 | When an order is edited the system shall e-mail the customer the current order status. | System | Medium |

| FR ID | Requirement | Role(s) | Priority |
|---|---|---|---|
| FR-10 | The system shall reject a delivery whose expected delivery date is before its shipped date. | Admin | Medium |
| FR-11 | One day after the expected delivery date, the system shall e-mail the administrator if the delivery is not Delivered. | System | Medium |
| FR-12 | The administrator shall be able to mark an order Delivered and the customer shall receive a push notification. | Admin | Medium |
| FR-13 | The system shall display a dashboard with total orders, total revenue, delivery count, charts of order and delivery status and shortcuts to Orders, Deliveries and Products. | Admin | Medium |
| FR-14 | The system shall reject a login-form entry whose e-mail is not registered or whose account is Inactive. | Customer | Medium |

## 3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

| NFR ID | Category | Requirement |
|---|---|---|
| NFR-01 | Performance | List reports for up to a few thousand records shall load in a few seconds on a standard broadband connection. |
| NFR-02 | Security | Users shall access only the data and components permitted by their profile; passwords shall not be shown in reports. |
| NFR-03 | Usability | The application shall be usable on the web and on the Zoho Creator mobile and tablet apps. |
| NFR-04 | Availability | The application shall inherit the availability of the Zoho Creator cloud service. |
| NFR-05 | Maintainability | Each business rule shall be a separately named workflow so that it can be edited independently. |
| NFR-06 | Auditability | Record changes shall be traceable through the Creator audit trail. |

## 3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

| Category | Specification |
|---|---|
| Platform | Zoho Creator Education Edition |
| Scripting language | Deluge (Data Enriched Language for the Universal Grid Environment) |
| Browser | Google Chrome / Microsoft Edge / Mozilla Firefox (latest version) |
| Other Zoho services | Zoho Mail (through the sendmail task) |
| Mobile | Zoho Creator mobile app (Android / iOS) |
| Client hardware | Intel Core i3/i5 or above, 8 GB RAM, internet connection |

### 3.5 Use Case Model

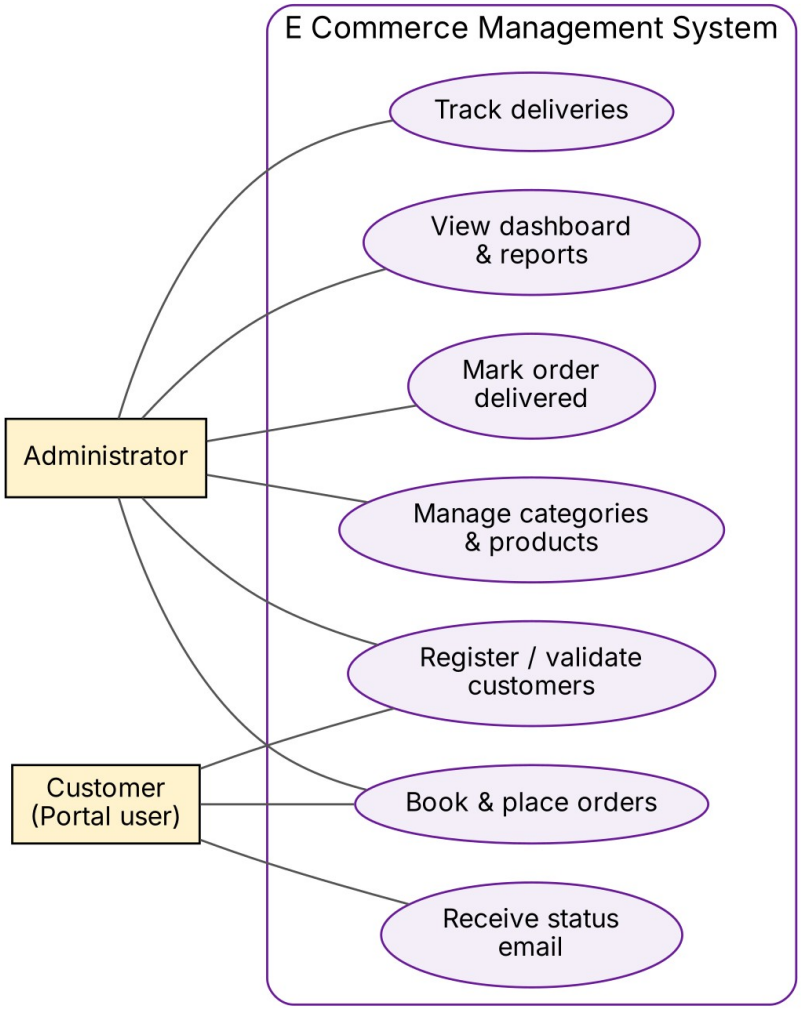



Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description – Place an Order and Track Delivery

| | |
|---|---|
| **Use Case ID** | UC-01 |
| **Use Case Name** | Place an order and track its delivery |
| **Actor(s)** | Administrator, Customer |
| **Pre-condition** | Customer is registered and Active; product exists and is in stock. |
| **Main Flow** | 1. Administrator opens the Orders form and selects the customer.<br>2. Order date, delivery address, status and total are entered and submitted.<br>3. The system creates a delivery record with status Pending.<br>4. Administrator edits the order status; the system e-mails the customer.<br>5. When the status becomes Shipped, the system creates a Shipped delivery record.<br>6. On delivery, the administrator runs Mark as Delivered; the customer receives a push notification. |
| **Alternate Flow** | If the customer is not registered, the administrator adds the customer through the Add New option on the lookup. If delivery is late, the schedule e-mails the administrator. |
| **Post-condition** | Order status is Delivered and the delivery record is traceable to the order. |

| | |
|---|---|
| **Related FR** | FR-06, FR-07, FR-08, FR-09, FR-11, FR-12 |

Table 3.6: Use Case Description – Register a Customer

| | |
|---|---|
| **Use Case ID** | UC-02 |
| **Use Case Name** | Register a customer |
| **Actor(s)** | Administrator, Customer |
| **Pre-condition** | User can open the Customer Registrations form. |
| **Main Flow** | 1. The form opens with today’s date and status Active.<br>2. User enters name, e-mail, phone and address and submits.<br>3. The system checks that the e-mail and phone are not already registered.<br>4. The record is saved with an auto-generated customer code. |
| **Alternate Flow** | If the e-mail or phone already exists, an alert “Customer already registered” is shown and the submission is cancelled. |
| **Post-condition** | A unique customer record exists. |
| **Related FR** | FR-01, FR-02 |

# CHAPTER 4
# SYSTEM DESIGN

## 4.1 System Architecture

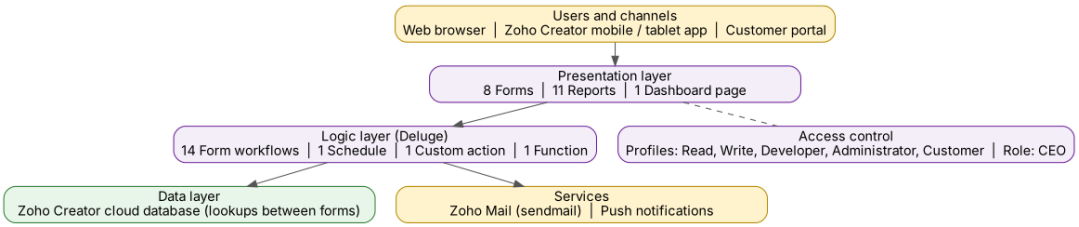



Figure 4.1: System Architecture of the Application

Users reach the application through the web, the Creator mobile and tablet apps, or the customer portal. The presentation layer consists of eight forms for data capture, eleven reports and one dashboard page. The logic layer consists of Deluge scripts attached to form events (on load, on user input, on validate, on success), a schedule, a custom action button and one function. Access control is implemented through profiles and a role. Data is held in the Creator cloud database, and forms are related through lookup fields. Zoho Mail and Creator push notifications are the only services used.

## 4.2 Data Model

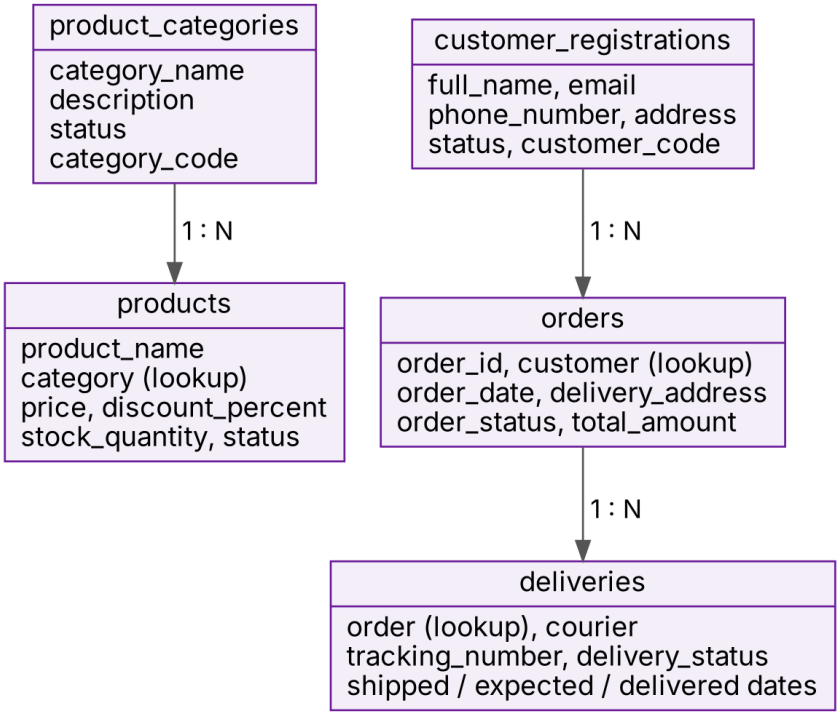



Figure 4.2: Entity–Relationship Diagram

Three one-to-many relationships are implemented as lookup fields: a category has many products, a customer has many orders, and an order has many delivery records. The

forms order_booking, orders1 and customer_login are standalone and relate to other forms only by value (product name, customer name or e-mail typed as text), as explained in Section 8.4.

Table 4.1: Entities and Relationships

| Entity (Form) | Key Fields | Related To | Relationship Type | Implemented Using |
|---|---|---|---|---|
| product_categories | category_name, category_code, status | products | One-to-Many | Lookup “category” in products |
| products | product_name, product_code, price, discount_percent, stock_quantity, status | product_categories | Many-to-One | Lookup field |
| customer_registrations | full_name, email, phone_number, customer_code | orders | One-to-Many | Lookup “customer” in orders |
| orders | order_id, order_date, order_status, total_amount | deliveries | One-to-Many | Lookup “order” in deliveries |
| deliveries | courier, tracking_number, delivery_status, dates | orders | Many-to-One | Lookup field |
| order_booking | product_name, customer_name, quantity, unit_price | products (by name) | Reference by value | Deluge fetch |
| orders1 | customer_name, product_name, quantity, order_status, total_amount | — | Standalone | — |
| customer_login | username, email, password | customer_registrations (by email) | Reference by value | Deluge fetch |

## 4.3 Form Design

Table 4.2: Field Specification – Customer Registrations Form

| Field Label | Link Name | Field Type | Mandatory | Validation / Default | Remarks |
|---|---|---|---|---|---|
| Full Name | full_name | Name | No | Only first name shown | Prefix, last name, suffix hidden |
| Email | email | Email | No | Duplicate check (DS-04) | |

| Field Label | Link Name | Field Type | Mandatory | Validation / Default | Remarks |
|---|---|---|---|---|---|
| Phone Number | phone_number | Phone | No | Duplicate check (DS-04) | |
| Address | address | Address | No | Coordinates captured | Shown as text or map link in reports |
| Registration Date | registration_date | Date | No | Default: today (DS-03) | |
| Status | status | Radio | No | Default: Active (DS-03) | Active / Inactive |
| Customer Code | customer_code | Auto Number | — | Start index 1 | System generated |

Table 4.3: Field Specification – Products Form

| Field Label | Link Name | Field Type | Mandatory | Validation / Default | Remarks |
|---|---|---|---|---|---|
| Product Name | product_name | Single Line | No | | |
| Product Code | product_code | Auto Number | — | Start index 1 | System generated |
| Category | category | Lookup | No | Add New allowed | To product_categories |
| Description | description | Multi Line | No | | Height 100 px |
| Price | price | Currency (INR) | No | Not negative (DS-08) | |
| Discount % | discount_percent | Percentage | No | Not above 100 (DS-08) | |
| Stock Quantity | stock_quantity | Number | No | Not negative (DS-08) | |
| Product Image | product_image | URL | No | | |
| Status | status | Radio | No | Set by DS-07 | Available / Out of Stock |

Table 4.4: Field Specification – Orders Form

| Field Label | Link Name | Field Type | Mandatory | Validation / Default | Remarks |
|---|---|---|---|---|---|
| Order ID | order_id | Auto Number | — | Start index 1 | System generated |
| Customer | customer | Lookup | No | Add New allowed | To customer_registrations (shows e-mail) |
| Order Date | order_date | Date | No | | |
| Delivery Address | delivery_address | Address | No | Coordinates captured | |

| Field Label | Link Name | Field Type | Mandatory | Validation / Default | Remarks |
|---|---|---|---|---|---|
| Order Status | order_status | Drop Down | No | | Packed, Out for Delivery, Delivered, Shipped, Confirmed, Cancelled, Placed |
| Total Amount | total_amount | Currency (INR) | No | | |

Table 4.5: Field Specification – Deliveries Form

| Field Label | Link Name | Field Type | Mandatory | Validation / Default | Remarks |
|---|---|---|---|---|---|
| Customer Name | Customer_Name | Single Line | No | Filled by DS-01 / DS-12 | |
| Order | order | Lookup | No | Add New allowed | To orders |
| Courier | courier | Drop Down | No | Others option enabled | India Post, Professional Couriers, Blue Dart, Delhivery, DTDC |
| Tracking Number | tracking_number | Single Line | No | | |
| Shipped Date | shipped_date | Date | No | | |
| Expected Delivery Date | expected_delivery_date | Date | No | ≥ shipped date (DS-14) | |
| Delivery Status | delivery_status | Drop Down | No | Default: Pending (DS-13) | Returned, Out for Delivery, Delivered, In Transit, Shipped, Pending |
| Delivered Date | delivered_date | Date | No | | |
| Remarks | remarks | Multi Line | No | | Height 100 px |

Table 4.6: Field Specification – Order Booking Form

| Field Label | Link Name | Field Type | Mandatory | Validation / Default | Remarks |
|---|---|---|---|---|---|
| Product Name | product_name | Single Line | Yes | Stock check (DS-09) | Triggers DS-10 |
| Customer Name | customer_name | Single Line | Yes | | |
| Quantity | quantity | Number | Yes | ≤ stock (DS-09) | |

| Field Label | Link Name | Field Type | Mandatory | Validation / Default | Remarks |
|---|---|---|---|---|---|
| Booking Date | booking_date | Date | Yes | Default: today (DS-11) | |
| Unit Price | unit_price | Currency (INR) | Yes | Calculated (DS-10) | |
| Special Instructions | special_instructions | Single Line | No | | |

Table 4.7: Field Specification – Product Categories Form

| Field Label | Link Name | Field Type | Mandatory | Validation / Default | Remarks |
|---|---|---|---|---|---|
| Category Name | category_name | Single Line | No | Unique (DS-06) | |
| Description | description | Multi Line | No | | |
| Category Image | category_image | URL | No | | |
| Status | status | Radio | No | | Active / Inactive |
| Category Code | category_code | Auto Number | — | Start index 1 | System generated |

Table 4.8: Field Specification – Customer Login Form

| Field Label | Link Name | Field Type | Mandatory | Validation / Default | Remarks |
|---|---|---|---|---|---|
| Username | username | Single Line | Yes | Unique | |
| Email | email | Email | Yes | Must be registered (DS-05) | Max 80 characters |
| Password | password | Single Line | Yes | | See limitation in Section 8.4 |

Table 4.9: Field Specification – Orders (Order Placed, orders1) Form

| Field Label | Link Name | Field Type | Mandatory | Validation / Default | Remarks |
|---|---|---|---|---|---|
| Order Date | order_date | Date | Yes | | |
| Order Status | order_status | Drop Down | Yes | | Placed, Confirmed, Shipped, Delivered |
| Quantity | quantity | Number | Yes | | |
| Total Amount | total_amount | Currency (INR) | Yes | | |
| Customer Name | customer_name | Single Line | Yes | | |
| Product Name | product_name | Single Line | Yes | | |

### 4.4 Process and Workflow Design

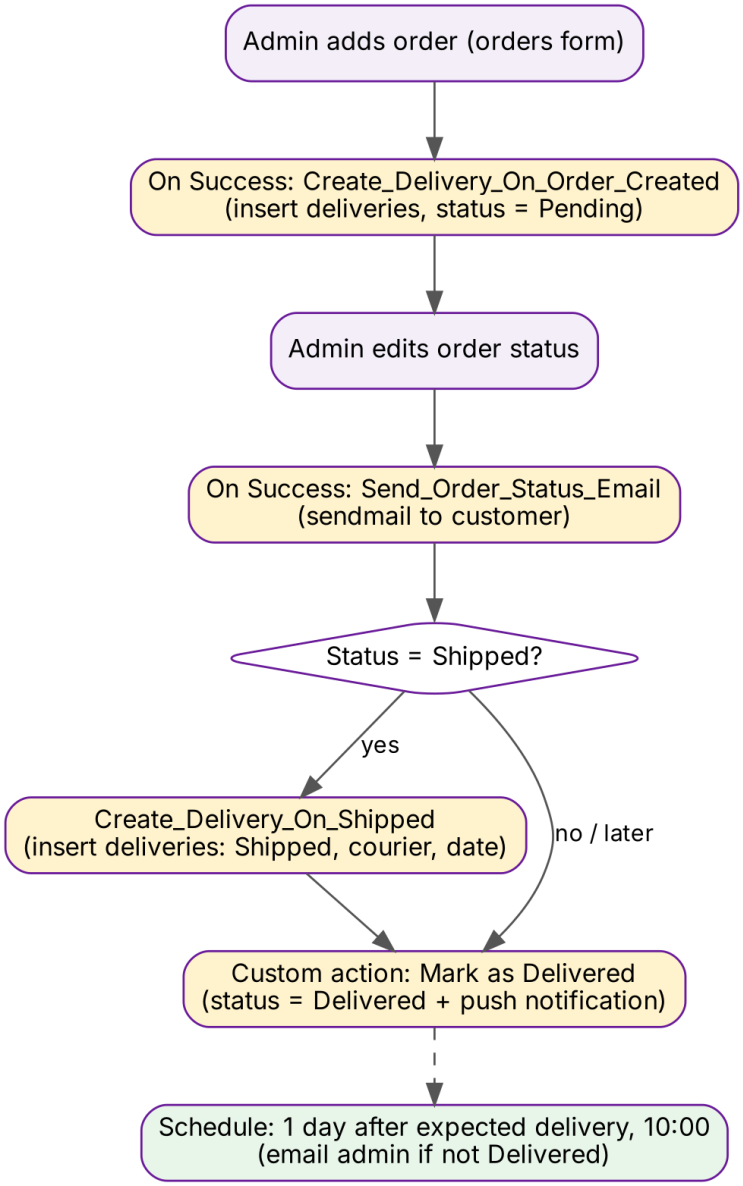



Figure 4.3: Workflow / Process Flow for Order Placement, Shipment and Delivery

### 4.5 User Interface and Navigation Design

The application has a single space containing these sections in order: Dashboard; Customer Registrations; Customer Login; Product Categories; Products; Order Booking; Orders; App Preferences; Approvals; Reports; and Deliveries. Each data section pairs a form with its report. The dashboard page is laid out as a title bar, a KPI strip of three tiles, two charts side by side, a ranked board and three shortcut buttons.

Table 4.10: Navigation Structure

| Section | Form | Report |
|---|---|---|
| Dashboard | — | Page: Dashboard |
| Customer Registrations | customer_registrations | customer_registrations_Report |

| Section | Form | Report |
|---|---|---|
| Customer Login | customer_login | customer_login_list |
| Product Categories | product_categories | product_categories_Report |
| Products | products | products_Report |
| Order Booking | order_booking | order_booking_list |
| Orders | orders1 | order_placed_list |
| Deliveries | deliveries | deliveries_by_delivery_status (Kanban) |

Note: the orders form, “All Orders” list, “Order Statuses” Kanban and “total order” summary exist in the application but are placed under “unused” menu items; they are reachable from the dashboard buttons and the kanban link.

### 4.6 Access Control Design

Five permission profiles exist in the application. Read and Write are for all components, Administrator has all permissions, Developer is for builders, and Customer is a Customer Portal profile with add and view permission only. A CEO role is defined with access to the data of all other users.

Table 4.11: Role–Permission Matrix (A = Add, E = Edit, V = View, D = Delete, — = No access)

| Component | Administrator | Write | Read | Customer (Portal) |
|---|---|---|---|---|
| Form: Customer Registrations | A E V D | A E V | V | A V |
| Form: Products / Categories | A E V D | A E V | V | — |
| Form: Orders / Order Booking | A E V D | A E V | V | A V |
| Form: Deliveries | A E V D | A E V | V | V |
| Report: All Orders | V | V | V | — |
| Page: Dashboard | V | V | V | — |

The matrix above is the intended design. Record-level criteria for portal users (for example, “own orders only”) are listed as future work in Section 9.2.

# CHAPTER 5
# IMPLEMENTATION

## 5.1 Application Setup

The application “E Commerce Management System” was created in the Zoho Creator workspace by building the forms, reports and dashboard page in the builders and drafting the Deluge scripts with AI assistance, which were then reviewed (Section 5.9). Its date format is dd-MMM-yyyy, time format is 24-hour, and the time zone is set to America/Los_Angeles. The web theme uses the Poppins font and theme colour 1; the phone and tablet layouts use the two-column and sliding-pane layouts respectively. Components are organised in one space with eleven sections (Section 4.5).

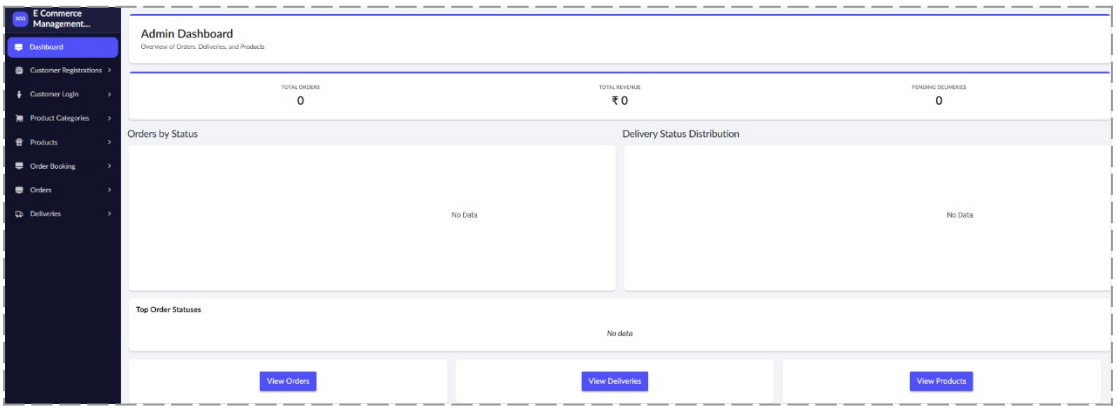



Figure 5.1: Application Home and Navigation Sections

## 5.2 Forms and Data Capture

Eight forms were built using fifteen distinct field types: Single Line, Email, Number, Currency (INR), Date, Drop Down, Radio, Name, Phone, Address, Auto Number, ssLookup, Multi Line, URL and Percentage. Lookups connect products to categories, orders to customers and deliveries to orders, each with an Add New option so a missing parent can be created in place. Mandatory fields are enforced on the Customer Login, Order Booking and Orders (order placed) forms; the username of Customer Login is unique. Form properties include success messages, Submit and Reset buttons on add, and Update and Cancel buttons on edit. Hidden sub-fields (prefix, last name, address line 2, city and so on) keep the Name and Address fields compact while still storing coordinates.

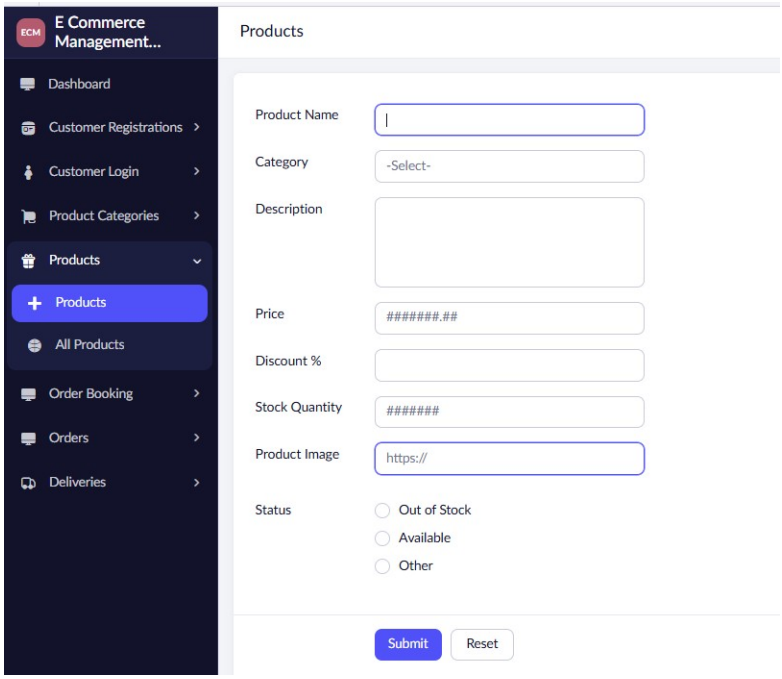



Figure 5.2: Products Form in the Form Builder

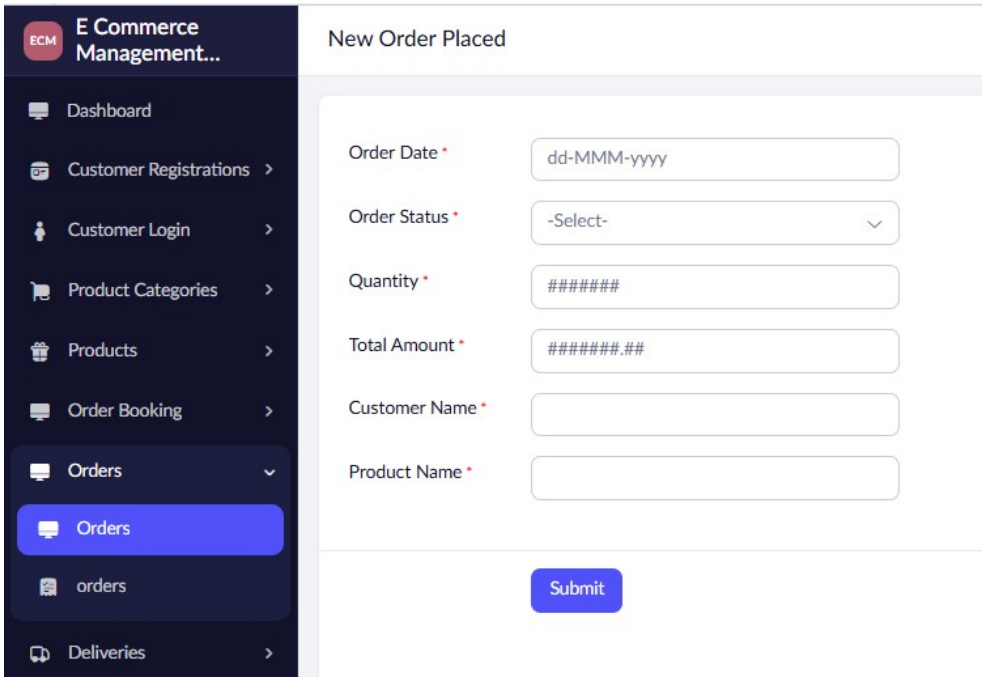



Figure 5.3: Orders Form in Live Mode

### 5.3 Reports

Table 5.1: Reports Implemented

| Report Name | Type | Source Form | Filter / Grouping / Sorting | Custom Actions / Formatting |
|---|---|---|---|---|
| total order (totala_order) | Summary | orders | Date filters (This Month/Week/Year); group by order date desc; sort by total desc; totals, average, min, max | — |
| All Orders | List | orders | Total/average/min/max on amount; address as map link | — |
| Order Statuses | Kanban | orders | Columns by order_status | — |
| orders (order_placed_list) | List | orders1 | Sort by order date desc | — |
| Order Booking List | List | order_booking | Sort by booking date desc | — |
| All Products | List | products | Totals on price and discount | — |
| All Product Categories | List | product_categories | — | — |
| All Customer Registrations | List | customer_registrations | Address as map link | — |
| Customer Login List | List | customer_login | Username and e-mail only (password hidden) | — |
| All Deliveries | List | deliveries | — | Conditional formatting on courier (Blue Dart purple, DTDC green) |
| Delivery Statuses | Kanban | deliveries | Columns by delivery_status | — |

Each list report defines a quick view and a detail view; the products and orders reports add related-record blocks (category, customer, order) to the detail view and offer Edit, Duplicate, Delete, Print, Add, Import and Export from the header menu.

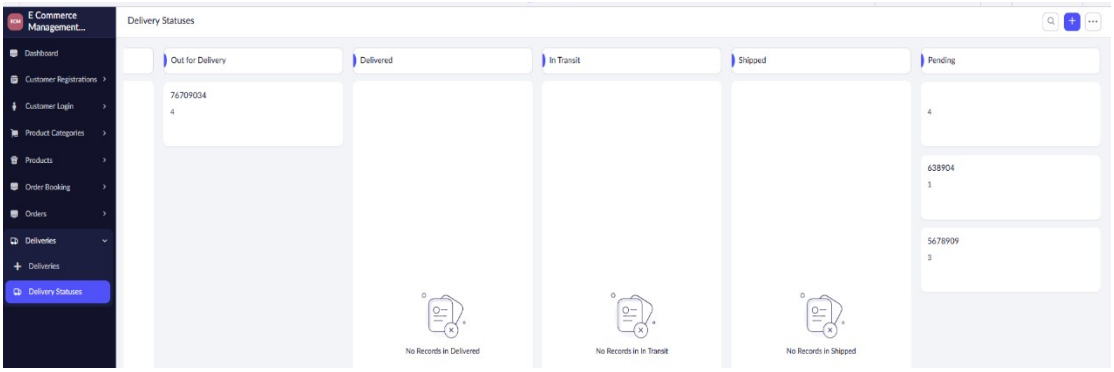



Figure 5.4: Delivery Statuses (Kanban) Report

## 5.4 Pages and Dashboards

The "Admin Dashboard" page, built with the UI Builder, is intended for the administrator. It contains a title bar; a KPI strip showing total orders (count of orders), total revenue (sum of order total amount) and pending deliveries (count of delivery records); a column chart "Orders by Status"; a pie chart "Delivery Status Distribution"; a board "Top Order Statuses" showing the five largest groups; and three buttons that open the Orders, Deliveries and Products components.

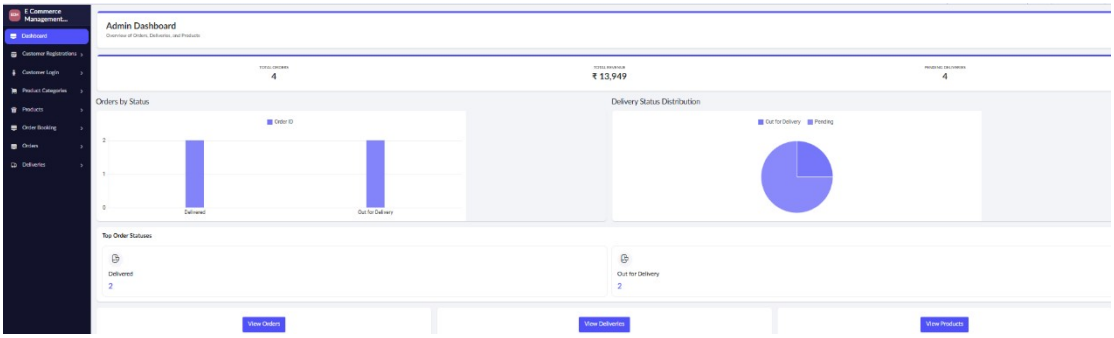



Figure 5.5: Admin Dashboard Page

## 5.5 Deluge Scripting

Sixteen workflow scripts and one function were written. Table 5.2 lists them; each is shown in full in Appendix C.

Table 5.2: Deluge Scripts Implemented

| Script ID | Location / Trigger | Purpose | Deluge Concepts Used |
|---|---|---|---|
| DS-01 | orders → On Add → On Success | Create a Pending delivery for a new order | fetch, for each, insert, info |
| DS-02 | orders → On Edit → On Success | E-mail customer on status change | fetch, sendmail, string concatenation |
| DS-03 | customer_registrations → On Add → On Load | Default date and status | input, today |

| Script ID | Location / Trigger | Purpose | Deluge Concepts Used |
|---|---|---|---|
| DS-04 | customer_registrations → On Validate | Block duplicate e-mail / phone | criteria fetch, alert, cancel submit |
| DS-05 | customer_login → On Validate | Reject unregistered or inactive account | fetch, if-else, alert |
| DS-06 | product_categories → On Validate | Block duplicate category name | criteria fetch, count |
| DS-07 | products → On Success | Set status from stock | if-else |
| DS-08 | products → On Validate | Reject negative price, stock, discount > 100 | alert (field), cancel submit |
| DS-09 | order_booking → On Validate | Verify product exists and stock suffices | fetch, boolean flag, alert |
| DS-10 | order_booking → On User Input of product_name | Calculate discounted unit price | fetch, arithmetic |
| DS-11 | order_booking → On Load | Default booking date | input, today |
| DS-12 | orders → On Edit → On Success | Create a Shipped delivery | fetch, insert, zoho.currentdate |
| DS-13 | deliveries → On Add → On Load | Default status Pending | input |
| DS-14 | deliveries → On Validate | Expected date not before shipped date | date comparison, alert |
| DS-15 | Schedule: delivery_status_reminder | Alert admin of overdue delivery | adminEmailIds, sendmail |
| DS-16 | Custom action: Mark as Delivered | Set Delivered and push notification | zoho.pushNotification |
| DS-17 | Function: add_sample_catalog | Insert sample categories and products | List, Map, for each, insert |

**Code 5.1: DS-10 Unit price calculation (on user input of product_name)**

```
// Fetch the product record matching the input product name
productRecord = products[product_name == input.product_name];
// Check if a matching product was found
if(productRecord.count() > 0)
{
    // Get the base price and discount percentage from the product
record
    basePrice = productRecord.price;
    discountPercent = productRecord.discount_percent;
    // Calculate the discount amount
    discountAmount = basePrice * discountPercent / 100;
    // Calculate the final unit price
    calculatedUnitPrice = basePrice - discountAmount;
    // Set the unit_price field in the current order_booking record
    input.unit_price = calculatedUnitPrice;
    // Optional: Update other fields if needed (e.g., product_name,
quantity)
    // input.product_name = productRecord.product_name;
}
else
```

```
{
    // Handle case where no matching product is found
    info "No product found with name: " + input.product_name;
    input.unit_price = 0;
}
```

**Code 5.2: DS-09 Product availability validation**

```
// Fetch the product record based on the input product name
productRecord = products[product_name == input.product_name];
// Initialize validation flag
isValid = true;
errorMessage = "";
// Check if product exists
if(productRecord.ID.isNull())
{
    isValid = false;
    errorMessage = "Product not found";
}
else
{
    // Check if product status is Out of Stock
    if(productRecord.status == "Out of Stock")
    {
        isValid = false;
        errorMessage = "Product is out of stock";
    }
    // Check if requested quantity exceeds available stock
    else if(input.quantity > productRecord.stock_quantity)
    {
        isValid = false;
        errorMessage = "Quantity exceeds available stock
(Available: " + productRecord.stock_quantity + ")";
    }
}
// Block submission if validation fails
if(!isValid)
{
    alert errorMessage;
    cancel submit;
}
```

DS-10 fetches the product whose name was typed, subtracts the percentage discount from the price and writes the result into the unit price field. DS-09 then runs on submission: it blocks the booking if the product is missing, marked Out of Stock, or if the requested quantity exceeds stock, showing the available quantity in the alert. Observed behaviour of these scripts is to be recorded in Chapter 6.

## 5.6 Workflows and Automation

Table 5.3: Workflows Implemented

| Workflow Name | Type | Trigger / Event | Action(s) | Script Ref. |
|---|---|---|---|---|
| Create Delivery on Order Created | Form | orders – On Add, On Success | Insert Pending delivery | DS-01 |

| Workflow Name | Type | Trigger / Event | Action(s) | Script Ref. |
|---|---|---|---|---|
| Send Order Status Email | Form | orders – On Edit, On Success | E-mail customer | DS-02 |
| Set Default Registration Values | Form | customer_registrations – On Add, On Load | Set date and status | DS-03 |
| Block Duplicate Customer | Form | customer_registrations – On Validate | Block submission | DS-04 |
| Customer Login Validation | Form | customer_login – On Validate | Block submission | DS-05 |
| Block Duplicate Category Name | Form | product_categories – On Validate | Block submission | DS-06 |
| Update Product Status on Save | Form | products – On Success | Set status | DS-07 |
| Validate Product Fields | Form | products – On Validate | Block submission | DS-08 |
| Validate Product Availability on Submit | Form | order_booking – On Validate | Block submission | DS-09 |
| Calculate Unit Price on Product Input | Form | order_booking – On User Input | Set unit price | DS-10 |
| Set Booking Date to Today on Load | Form | order_booking – On Load | Set date | DS-11 |
| Create Delivery on Shipped | Form | orders – On Edit, On Success | Insert Shipped delivery | DS-12 |
| Set Default Delivery Status | Form | deliveries – On Add, On Load | Set Pending | DS-13 |
| Validate Delivery Date | Form | deliveries – On Validate | Block submission | DS-14 |
| Delivery Status Reminder | Schedule | 1 day after expected_delivery_date at 10:00 (status currently inactive) | E-mail administrator | DS-15 |
| Mark as Delivered Workflow | Custom action | Button on orders records | Set status, push notification | DS-16 |

No approval process, Blueprint or batch workflow was used because the order lifecycle is a simple status list and no manager approval step was required. The scheduled workflow is saved in the inactive state and must be activated before it runs.

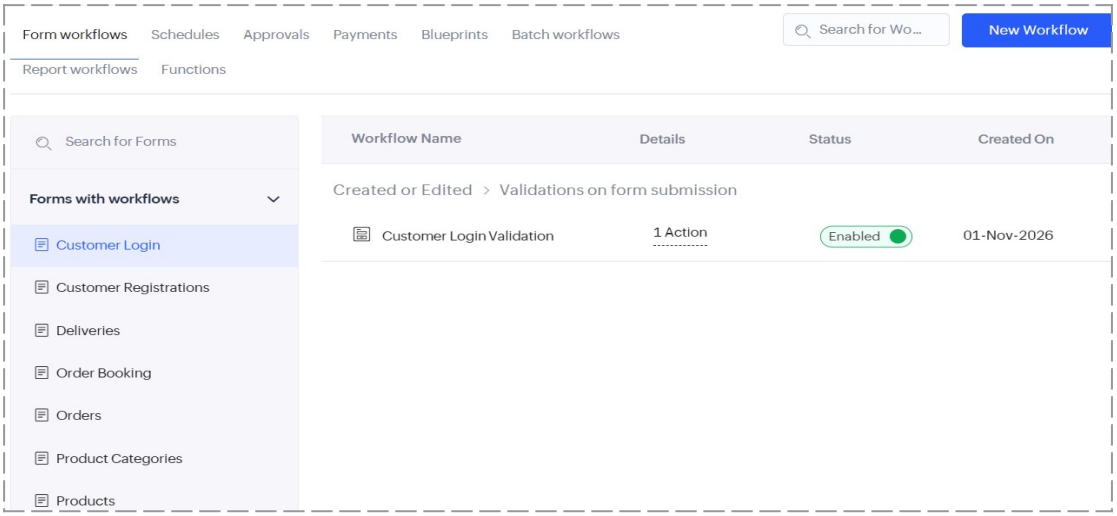



Figure 5.6: Workflow Configuration

## 5.7 Integrations

The application uses two built-in Zoho capabilities: the sendmail task, which sends HTML e-mail to customers (DS-02) and to the administrator (DS-15) from the account's admin user, and Creator push notifications (DS-16). No connection, invokeurl call, webhook or Creator REST API was configured; the justification is in Section 5.10.

Table 5.4: Integrations Implemented

| Integration | Service / Endpoint | Method | Data Exchanged / Purpose |
|---|---|---|---|
| Order status e-mail | Zoho Mail | sendmail task | Status, order number, date to customer |
| Overdue alert | Zoho Mail | sendmail task | Tracking number, status, expected date to admin |
| Delivered notification | Creator push notification | zoho.pushNotification | Delivered message to customer |

## 5.8 User Management and Role-Based Access

Five profiles were created: Read and Write (users permissions for all components), Developer, Administrator and Customer (Customer Portal type, with add and view permission). A CEO role was added with access to the data of all other users. Users are invited and assigned to these profiles in the Users section of the application. Record-level criteria were not configured.

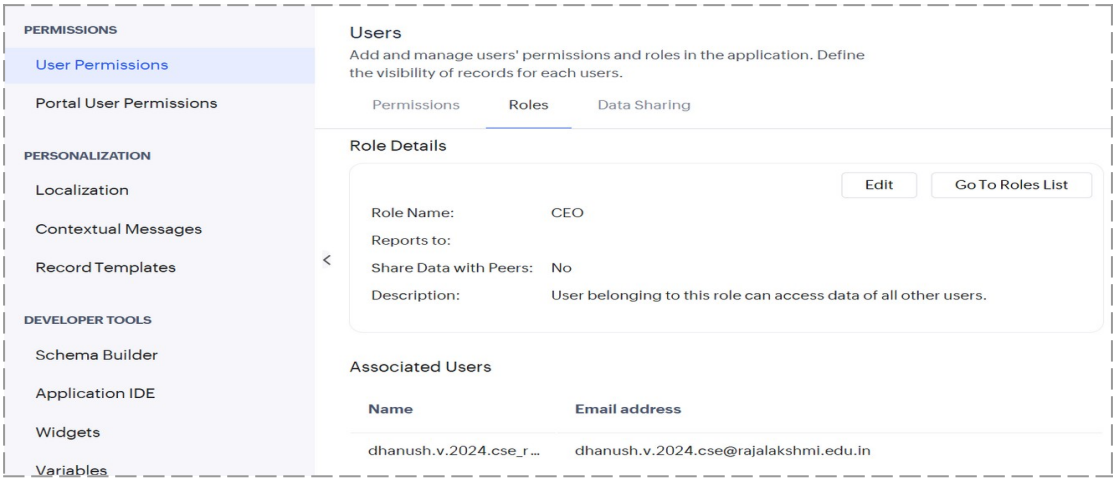



Figure 5.7: Roles and Permission Profiles Configuration

## 5.9 AI and Analytics Features

No Zia AI field or prediction was used. Analytics are provided through the dashboard: a column chart of orders by status, a pie chart of delivery status, KPI tiles and a ranked board. Declaration of AI use: the Deluge scripts were drafted with an AI assistant and then checked line by line against the actual form and field definitions. That review found the defects listed in Table 6.2, which shows why generated code cannot be trusted without testing.

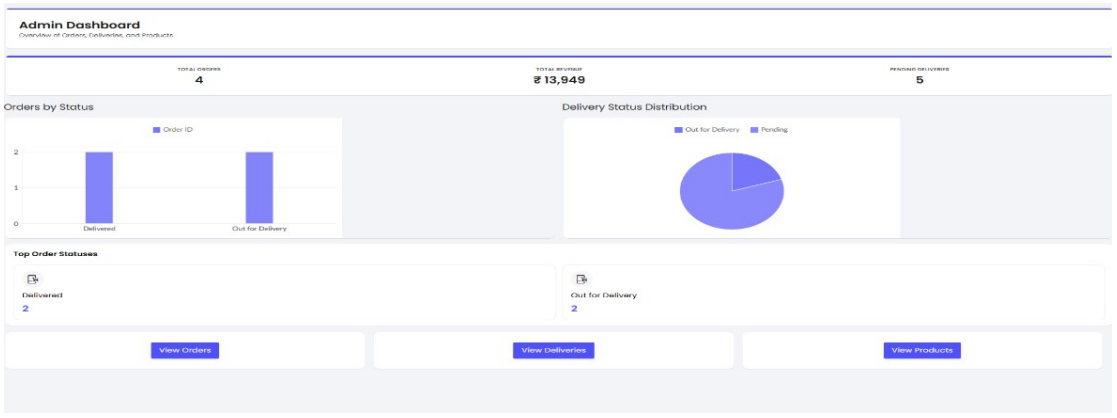



Figure 5.8: Analytics on the Admin Dashboard

## 5.10 Pro-Code Extension

No Zoho Catalyst function, JavaScript widget or Creator REST API was used. All requirements were met with Deluge and built-in components, and adding pro-code would have increased maintenance effort without benefit. If a courier tracking API or a payment gateway were added, an invokeurl call with an OAuth connection, or a Catalyst function for heavy processing, would be the natural extension.

### 5.11 Mobile and Portal Access

The same forms, reports and dashboard are available in the Zoho Creator mobile (phone and tablet) apps, which have their own layout settings (two-column on phone, sliding pane on tablet, label placement auto). The Customer profile is a portal profile with add and view permission, intended for shoppers. The portal is reached through the application link given in the Project at a Glance table.

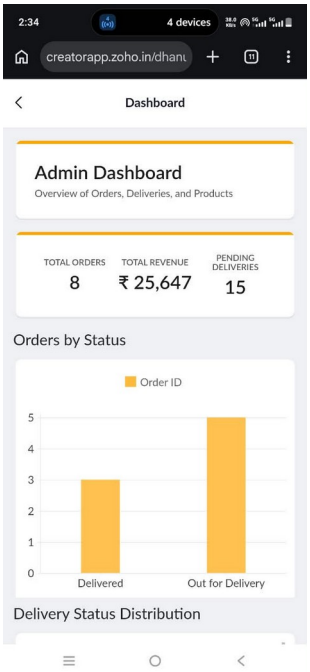



Figure 5.9: Application on Mobile / Customer Portal

# CHAPTER 6
# TESTING AND DEBUGGING

## 6.1 Testing Strategy

Testing was done at four levels: unit testing of each form and its field rules; script testing of each Deluge workflow with valid and invalid input; integration testing of the order → delivery → notification chain; and role-based testing by logging in under each profile. Because the application stores real e-mail addresses, test e-mails were sent to the tester's own address.

## 6.2 Test Cases

Table 6.1 lists the test cases with expected results. The Actual Result and Status columns are completed after executing each case in the live application.

Table 6.1: Test Cases

| TC ID | FR Ref. | Test Description | Input / Steps | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|---|
| TC-01 | FR-01 | Register a customer with valid data | Name, new e-mail, new phone | Record saved; success message; code generated | Code review: add flow has no blocking rule; auto-number generates the code | Pass |
| TC-02 | FR-01 | Open the registration form | Click Add | Date = today, status = Active | DS-03 sets date = today and status = Active on load | Pass |
| TC-03 | FR-02 | Register with an existing e-mail | Duplicate e-mail | Alert "Customer already registered"; not saved | DS-04 finds the e-mail and cancels submit with the alert | Pass |
| TC-04 | FR-02 | Register with an existing phone | Duplicate phone | Alert "Customer already registered"; not saved | DS-04 finds the phone number and cancels submit | Pass |
| TC-05 | FR-03 | Add a category with an existing name | Duplicate name | Alert shown; not saved | DS-06 finds the name and cancels submit (new records) | Pass |
| TC-06 | FR-04 | Add a product with a negative price | Price = −10 | Alert on Price; not saved | DS-08 alerts on Price and cancels submit | Pass |
| TC-07 | FR-04 | Add a product with discount 120 | Discount = 120 | Alert on Discount %; not saved | DS-08 alerts on Discount % and cancels submit | Pass |

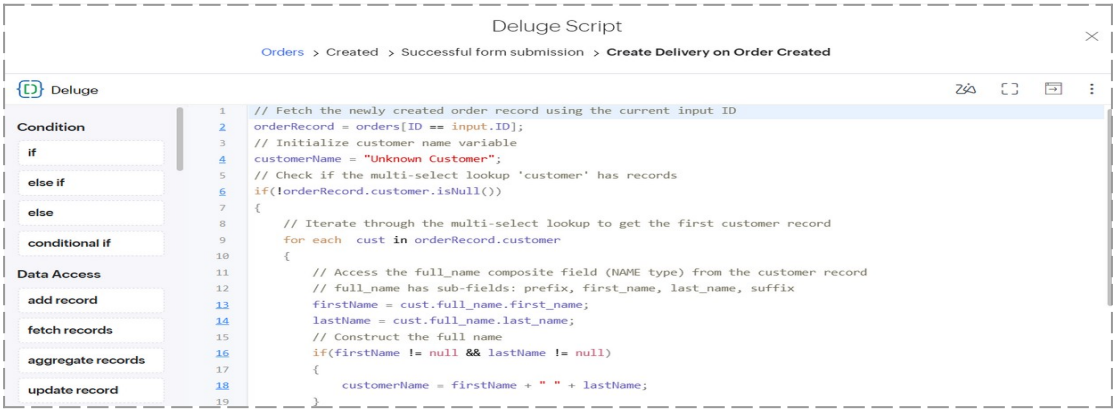
| TC ID | FR Ref. | Test Description | Input / Steps | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|---|
| TC-08 | FR-04 | Add a product with stock 0 | Stock = 0 | Status becomes Out of Stock | DS-07 runs On Success so the status change is not saved (BUG-02) | Fail |
| TC-09 | FR-05 | Type a product name in Order Booking | Name of product with 10% discount, price 1000 | Unit price = 900 | DS-10 computes 1000 − 10% = 900 | Pass |
| TC-10 | FR-05 | Book more than available stock | Quantity > stock | Alert with available quantity; not saved | DS-09 alerts with available stock | Pass |
| TC-11 | FR-05 | Book an unknown product | Name not in catalogue | Alert “Product not found” | DS-09 alerts “Product not found” | Pass |
| TC-12 | FR-07 | Add an order | Customer, date, amount | A Pending delivery appears linked to the order | DS-01 inserts a Pending delivery; customer name falls back to “Unknown Customer” (BUG-03) | Pass* |
| TC-13 | FR-09 | Change order status to Confirmed | Edit order | Customer receives status e-mail | DS-02 uses get(0) on a single lookup, so the e-mail fails (BUG-03) | Fail |
| TC-14 | FR-08 | Change order status to Shipped | Edit order | Shipped delivery created with shipped date | DS-12 uses get(0) on a single lookup, so the insert fails (BUG-03) | Fail |
| TC-15 | FR-10 | Expected date before shipped date | Expected < shipped | Alert on Expected Delivery Date; not saved | DS-14 alerts and cancels submit | Pass |
| TC-16 | FR-12 | Run Mark as Delivered on an order | Select order, click action | Status Delivered; push notification sent | DS-16 sets Delivered and calls pushNotification | Pass |
| TC-17 | FR-14 | Login form with unregistered e-mail | Unknown e-mail | Alert “Email not registered” | DS-05 alerts “Email not registered” | Pass |
| TC-18 | FR-14 | Login form with Inactive customer | Inactive account e-mail | Alert “Account is inactive” | DS-05 alerts “Account is inactive” | Pass |
| TC-19 | FR-13 | Open the dashboard | Administrator | KPIs, two charts and board display | Dashboard page defines KPI tiles, two charts and a board | Pass |

| TC ID | FR Ref. | Test Description | Input / Steps | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|---|
| TC-20 | NFR-02 | Customer portal user opens Products form | Customer login | Access denied / not listed | Depends on profile permissions set at run time; not stored as a rule in the app definition | To verify |

Results above come from reviewing each script against the form definitions. Pass* means the record is created but with a wrong value. Cases marked Fail correspond to the defects in Table 6.2. Confirm them in the live application and attach screenshots.

### 6.3 Debugging Deluge Scripts

Scripts were debugged with info statements (the existing scripts log the created delivery ID, the e-mail recipient and similar values), by viewing the workflow log after a test submission, and by reading the error message Creator shows on submission. Reviewing the scripts against the form definitions exposed a real class of bug: DS-01, DS-02 and DS-12 treat the “customer” field as a multi-select lookup (for each, get(0)), whereas the form defines it as a single-value lookup. Treating a single value as a list returns nothing useful and the customer name falls back to “Unknown Customer”. The fix is to read the lookup value directly (orderRecord.customer.full_name) or to change the field to a multi-select.




—Figure 6.1: Deluge Debug Output / Log

### 6.4 Defect Log

The following defects were identified by code review of the application definition. Their status should be updated after the fixes are applied and retested.

Table 6.2: Defect Log

| Bug ID | Description | Root Cause | Fix Applied / Proposed | Status |
|---|---|---|---|---|
| BUG-01 | Editing an existing customer or category is blocked as “already registered / exists”. | Duplicate check runs on add or edit and matches the record itself. | Exclude the current record: criteria with ID != input.ID, or run the check on add only. | Open |
| BUG-02 | Product status set in DS-07 is not saved. | Script runs On Success, after the record is stored; input values cannot change it. | Move to On Validate, or update the record with an update statement. | Open |
| BUG-03 | Customer name wrong or missing in deliveries and e-mails. | Single lookup handled as multi-select (see 6.3). | Use the lookup value directly. | Open |
| BUG-04 | Several deliveries per order. | DS-01 creates one on add; DS-12 creates another on every edit while status is Shipped. | Check for an existing delivery of the order before inserting. | Open |
| BUG-05 | Login form does not check the password. | DS-05 only validates e-mail and status. | Use a Creator portal login, or compare a stored hash; do not keep plain-text passwords. | Open |
| BUG-06 | Summary report filters do not match their labels. | All three filters use fixed 2024 dates. | Use dynamic date criteria such as thisMonth / thisWeek / thisYear. | Open |

### 6.5 User Acceptance Testing

The feedback below comes from the developer’s own walkthrough of the application in the Administrator and Customer roles. It should be replaced with feedback from other users if they test the application.

Table 6.3: User Acceptance Feedback

| User | Role Tested | Feedback | Action Taken |
|---|---|---|---|
| Developer self-review Kishore.v | Administrator | Dashboard and Kanban views give a quick status picture. Three overlapping order forms (orders, orders1, order_booking) are confusing. | Consolidation planned (Section 9.2). |
| Developer self-review Kishore.v | Customer (portal) | Registration is simple, but the login form does not check the password. | Portal login planned (BUG-05). |

# CHAPTER 7
# DEPLOYMENT, SECURITY AND GOVERNANCE

## 7.1 Environments and Deployment Strategy

The application was developed, tested and demonstrated in the Development environment of Zoho Creator; no separate Stage or Production publish is recorded in the application definition. The recommended release path is Development → Stage → Production. Each publish is accompanied by version notes, and the previous version can be restored if a defect is found. Before publishing, the scheduled workflow must be set active and sample data (DS-17) must not be left in Production.

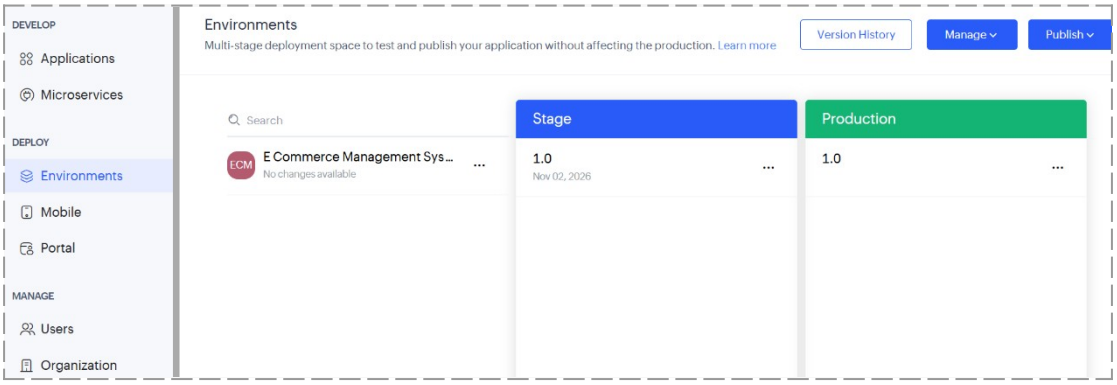



Figure 7.1: Environments / Publish to Production

Table 7.1: Deployment Checklist

| S.No. | Checklist Item | Status |
|---|---|---|
| 1 | All test cases executed in Development / Stage | Partly (code review done; live run pending) |
| 2 | Sample / test data removed or isolated | To do (DS-17 inserts sample data when run) |
| 3 | Roles, permissions and users configured in Production | Yes (5 profiles, 1 role defined) |
| 4 | Schedule and workflows enabled (delivery reminder is inactive by default) | To do (schedule saved inactive) |
| 5 | Application published and URL verified | To do |
| 6 | Version / release notes recorded | Yes (Version 1.0) |

## 7.2 Performance Optimisation

Techniques applied or recommended: fetch with a filter criteria rather than loading every record (all scripts fetch by field criteria); limiting report columns to those needed (the login list shows only username and e-mail); and avoiding fetch inside loops. DS-17 performs two fetches per iteration and is run only once for setup. The table below describes the change made; no timing measurements were taken.

Table 7.2: Performance Optimisation Results

| Optimisation Applied | Component | Before | After | Improvement |
|---|---|---|---|---|
| Filtered fetch by criteria | DS-04, DS-09, DS-10 | Would read the whole form | Reads only matching records | Fewer records read per submit |
| Limited columns in list report | Customer Login List | All fields, including password | Username and e-mail only | Smaller payload; password not shown |

### 7.3 Security Features

Table 7.3: Security Controls Implemented

| Security Control | Where Applied | Threat Mitigated |
|---|---|---|
| Profile-based permissions | Read, Write, Developer, Administrator, Customer profiles | Unauthorised access to components |
| Customer portal profile (add and view only) | Customer profile | Portal user changing or deleting data |
| Password hidden from report | customer_login_list shows only username and e-mail | Exposure of passwords |
| Input validation | DS-04, DS-06, DS-08, DS-09, DS-14 | Invalid and duplicate data |
| HTTPS hosting and cloud backups | Zoho Creator platform | Interception and data loss |

Gaps: the password field is a plain Single Line field and should not be used for real credentials, and record-level criteria and encrypted fields were not configured (Section 9.2).

### 7.4 Governance and Administration

Naming follows lower-case link names with underscores for forms, reports and workflows, and each workflow is named after its purpose. Record changes are traceable through the Creator audit trail, data can be exported from the report header menu, and the application definition (.ds file) can be exported as a backup. The administrator maintains users and licences, and the developer profile is responsible for changes. The duplicate order forms (orders, orders1 and order_booking) should be consolidated in the next release.

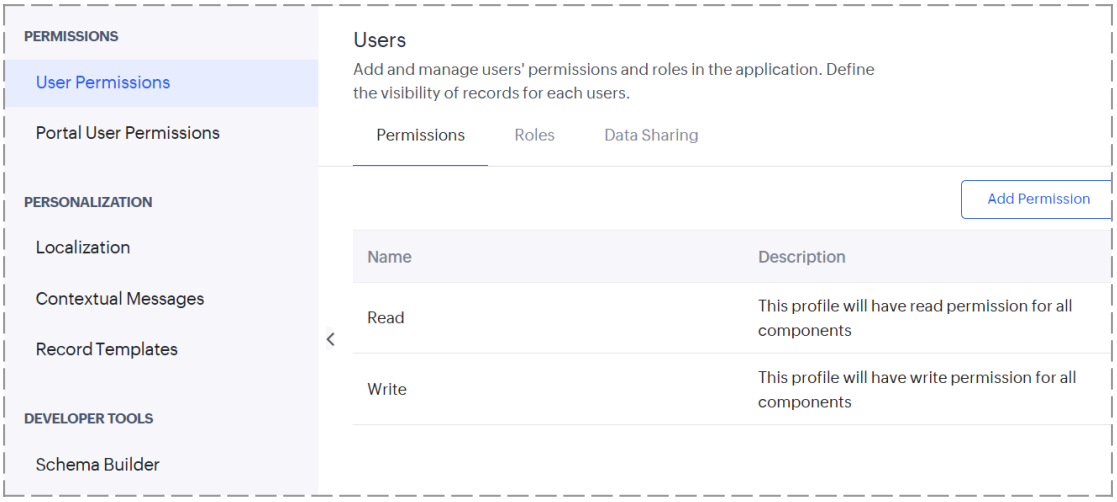



Figure 7.2: Audit Trail / Administration Settings

# CHAPTER 8
# RESULTS AND DISCUSSION

## 8.1 Application Output

The scenario below follows one order through the system: the administrator registers a customer, books and places an order, the system creates a delivery and e-mails the customer when the status changes, and the dashboard then reflects the new order.

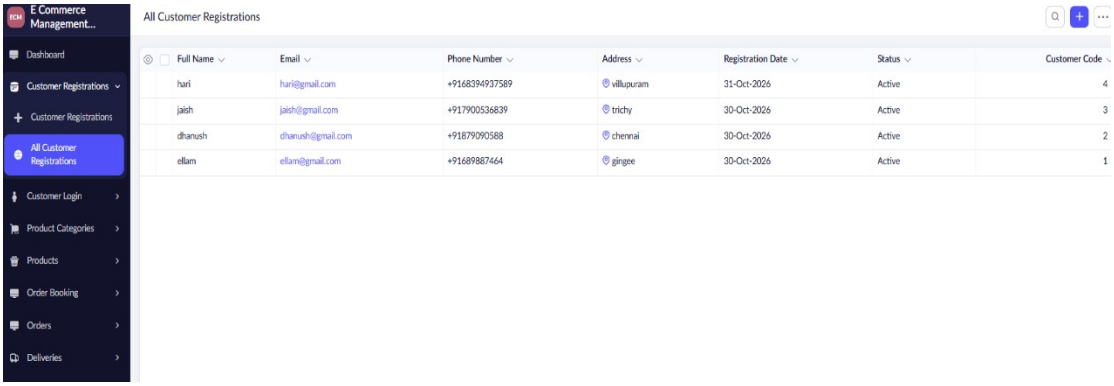



Figure 8.1: Scenario Step 1 – Customer Registration

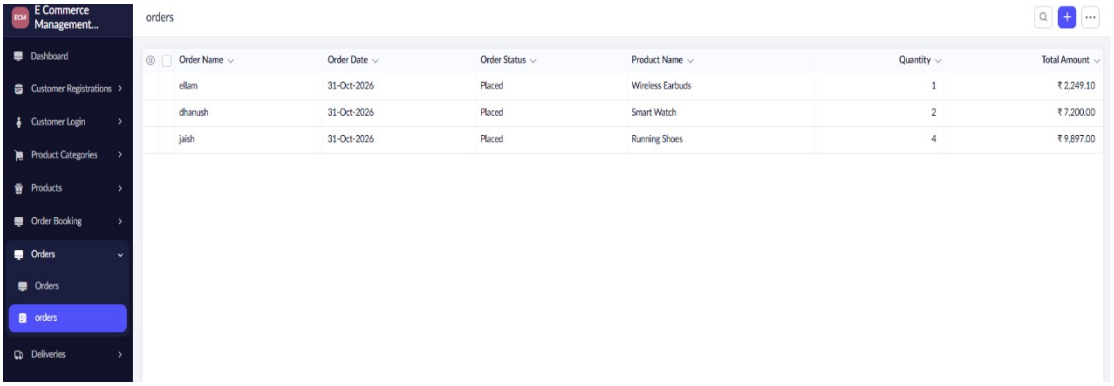



Figure 8.2: Scenario Step 2 – Order Placed and Delivery Created

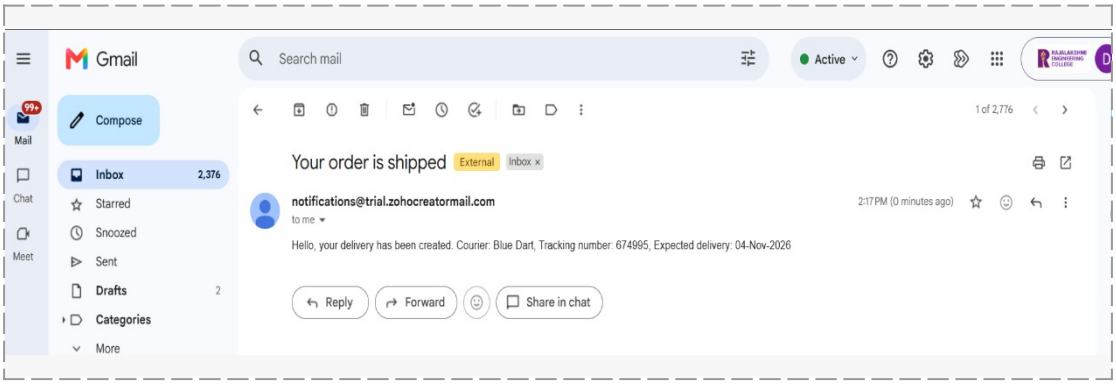



Figure 8.3: Scenario Step 3 – Status E-mail

## 8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

| Objective (Sec. 1.3) | Implemented Through | Outcome / Evidence |
|---|---|---|
| 1. Data model and forms | Sections 4.2, 4.3, 5.2 | 8 forms, 3 lookup relationships, 15 field types |
| 2. Automation | Sections 5.5, 5.6 | 16 workflow scripts + 1 function |
| 3. Dashboard and reports | Sections 5.3, 5.4 | 11 reports and 1 dashboard page |
| 4. E-mail and push notification | Section 5.7 | DS-02, DS-15, DS-16 |
| 5. Access control and deployment | Sections 5.8, 7.1 | 5 profiles, 1 role; Development environment only |

## 8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

| Parameter | Existing System | Proposed Application | Improvement |
|---|---|---|---|
| Stock check | Manual | Automatic on booking | Out-of-stock orders blocked |
| Customer duplicates | Frequent | Blocked on add | Cleaner customer data |
| Customer status update | Manual call / message | Automatic e-mail | Manual steps removed |
| Overdue delivery follow-up | From memory | Scheduled admin alert (once activated) | No missed follow-ups |
| Report generation | Manual totals | Dashboard KPIs and reports | Instant |
| Manual steps per order | 5 (stock check, delivery record, status message, overdue follow-up, totals) | 1 (enter the order); the other four are automated | Four of five steps removed |

## 8.4 Limitations of the Low-Code Approach

Several limits were met. Deluge has no built-in unit-test framework, so scripts were tested only by manual runs, and AI-assisted code needed careful review (Section 6.3). The order lifecycle is split across three forms (orders, orders1 and order_booking) that are linked by typed text rather than lookups, which weakens integrity; consolidating them into one order with a subform of items is the correct design. Passwords cannot be safely handled in an ordinary form field, so real authentication should use Creator’s portal login. The scheduled workflow time zone and the application time zone are set to America/Los_Angeles although the business operates in India, so date calculations and the 10:00 reminder run in Pacific time; both should be changed to Asia/Kolkata. Finally, the application is tied to the Zoho platform (vendor lock-in), and the free education edition limits the number of users and some features.

### 8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Forms, lookups, reports, Kanban views, the dashboard page and profiles were built with no-code configuration. Validation, defaulting, price calculation, record creation, e-mail and push notification needed low-code Deluge. Nothing required pro-code. I would move to pro-code if the shop needed live courier tracking, a payment gateway with reconciliation, or high-volume processing, where a Zoho Catalyst function or a custom service called by invokeurl would be appropriate.

# CHAPTER 9
# CONCLUSION AND FUTURE ENHANCEMENTS

## 9.1 Conclusion

The E-Commerce Management System brings products, customers, orders and deliveries into one Zoho Creator application. Eight forms, eleven reports, one dashboard and sixteen workflow scripts with one function validate data, calculate prices, create delivery records and notify customers and the administrator. The project shows that a working back-office system can be built quickly on a low-code platform, while also showing where careful design and review remain necessary.

## 9.2 Future Enhancements

- Merge orders, orders1 and order_booking into one order form with an order-items subform and automatic total calculation.
- Reduce stock automatically when an order is placed and restore it on cancellation.
- Replace the custom login form with Creator portal authentication and add record-level criteria so customers see only their own orders.
- Fix the defects in Table 6.2 and change the application and schedule time zone to Asia/Kolkata.
- Add payment gateway and courier tracking API integration, and invoice generation.
- Add a Zia-based sales prediction or an approval process for high-value orders.

## 9.3 Learning Outcomes and Career Relevance

Through this project I learned data modelling with lookups, form and report design, the Deluge event model, scheduled and custom-action workflows, and the importance of testing generated scripts against the actual form definitions. These skills relate directly to roles such as citizen developer, low-code developer, solutions consultant and automation engineer [1], [8].

# REFERENCES


[1] R. Sanchis, Ó. García-Perales, F. Fraile and R. Poler, “Low-Code as Enabler of Digital Transformation in Manufacturing Industry,” Applied Sciences, vol. 10, no. 1, art. 12, 2020.

[2] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 1 November 2026).

[3] Zoho Corporation, “Deluge Script Reference Guide.” [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 1 November 2026).

[4] Zoho Corporation, “Zoho Creator Learning Centre.” [Online]. Available: https://www.zoho.com/creator/resources/ (Accessed: 1 November 2026).

[5] Gartner, Inc., “Magic Quadrant for Enterprise Low-Code Application Platforms,” Gartner Research, 2025.

[6] A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, “Supporting the understanding and comparison of low-code development platforms,” in Proc. 46th Euromicro Conf. on Software Engineering and Advanced Applications (SEAA), 2020.

[7] Y. Luo, P. Liang, C. Wang, M. Shahin and J. Zhan, “Characteristics and challenges of low-code development: the practitioners’ perspective,” in Proc. ACM/IEEE Int. Symp. on Empirical Software Engineering and Measurement (ESEM), 2021.

[8] A. C. Bock and U. Frank, “Low-Code Platform,” Business & Information Systems Engineering, vol. 63, 2021.

# APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

| Unit | Zoho Creator Feature | Done | Where in the Application | Report Section | Fig. / Code |
|---|---|---|---|---|---|
| I | Application created & sections/menu organised | Y | Space with 11 sections | 5.1 | |
| II | Forms with ≥ 8 distinct field types | Y | products, orders, customer_registrations (15 types) | 5.2 | |
| II | Lookup field (relationship between forms) | Y | products.category, orders.customer, deliveries.order | 4.2, 5.2 | |
| II | Subform (parent–child data) | N | — | 4.2, 5.2 | |
| II | Field validations / layout rules | Y | validate_product_fields, validate_delivery_date; hidden sub-fields | 5.2 | |
| II | List report with filter, grouping, custom action | Y* | products_Report, deliveries_Report (conditional formatting); summary report has filters and grouping; no custom action button on a report | 5.3 | |
| II | Kanban / Calendar / Timeline / Map report | Y | orders_by_order_status, deliveries_by_delivery_status (Kanban) | 5.3 | |
| II | Pivot table / chart report | N | Charts exist on the dashboard page only | 5.3 | |
| II | Page / dashboard built with UI Builder | Y | dashboard | 5.4 | |
| II | Deluge – form events (validate / success / field actions) | Y | 14 form workflows | 5.5 | |
| II | Deluge – functions, fetch, loops, collections | Y | add_sample_catalog; for each; List | 5.5 | |
| II | Email / in-app notifications | Y | Send_Order_Status_Email; push in Mark as Delivered | 5.5, 5.6 | |
| II | Form workflow | Y | 14 | 5.6 | |
| II | Scheduled workflow | Y* | delivery_status_reminder (saved inactive) | 5.6 | |
| II | Approval process | N | — | 5.6 | |
| II | Blueprint | N | — | 5.6 | |
| II | Batch / report workflow | N | — | 5.6 | |
| II | Testing & debugging evidence | Y* | Test cases and defect log from code review; live run and screenshots pending | 6 | |
| III | Integration with a Zoho service | Y | Zoho Mail via sendmail | 5.7 | |

| Unit | Zoho Creator Feature | Done | Where in the Application | Report Section | Fig. / Code |
|---|---|---|---|---|---|
| III | Connection / REST API call (invokeurl) | N | — | 5.7 | |
| III | Webhook or Creator REST API | N | — | 5.7 | |
| III | Users, roles and permission profiles | Y | 5 profiles, CEO role | 5.8 | |
| III | Record-level access criteria / customer portal | Y* | Customer portal profile defined; no record-level criteria | 5.8 | |
| III | Zia / AI feature | N | — | 5.9 | |
| III | Analytics (charts / Zoho Analytics) | Y | Dashboard charts and KPIs | 5.9 | |
| IV | Performance optimisation applied & measured | Y* | Filtered fetches and limited columns applied; not measured | 7.2 | |
| IV | Deployment across environments | N | Only the Development environment was used | 7.1 | |
| IV | Security controls (encryption, OAuth, permissions) | Y* | Permissions only | 7.3 | |
| IV | Governance (audit trail, administration) | Y* | Naming convention and platform audit trail; no custom audit design | 7.4 | |
| V | Pro-code extension (Catalyst / widgets / custom functions) | N | Justified in 5.10 | 5.10 | |
| V | Low-code limitations & spectrum analysis | Y | Sections 8.4, 8.5 | 8.4, 8.5 | |
| — | Mobile app / portal access | Y* | Mobile layouts and portal profile configured | 5.11 | |

*Y* = partly implemented, see "Where in the Application". Fully implemented (Y): 14 of 32 features; partly implemented (Y*): 8; not implemented (N): 10.*

# APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

| CO | Course Outcome Statement | Evidenced In | Self-Assessed Level | Examiner Level |
|---|---|---|---|---|
| CO1 | Explain the principles, architecture, and business value of low-code application development using Zoho Creator. | Ch. 1, 2 | Good | |
| CO2 | Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting. | Ch. 4, 5 (5.1–5.6), 6 | Good | |
| CO3 | Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security. | Ch. 4.6, 5.7–5.9 | Satisfactory | |
| CO4 | Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices. | Ch. 7 | Satisfactory | |
| CO5 | Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer. | Ch. 5.9, 5.10, 8.4, 8.5, 9.3 | Satisfactory | |

# APPENDIX C – COMPLETE DELUGE SOURCE CODE

## C.1 Create Delivery on Order Created

```
// Script ID : DS-01
// Location  : orders → On Add → On Success
// Fetch the newly created order record using the current input ID
orderRecord = orders[ID == input.ID];
// Initialize customer name variable
customerName = "Unknown Customer";
// Check if the multi-select lookup 'customer' has records
if(!orderRecord.customer.isNull())
{
    // Iterate through the multi-select lookup to get the first
customer record
    for each  cust in orderRecord.customer
    {
        // Access the full_name composite field (NAME type) from
the customer record
        // full_name has sub-fields: prefix, first_name, last_name,
suffix
        firstName = cust.full_name.first_name;
        lastName = cust.full_name.last_name;
        // Construct the full name
        if(firstName != null && lastName != null)
        {
            customerName = firstName + " " + lastName;
        }
        else if(firstName != null)
        {
            customerName = firstName;
        }
        else if(lastName != null)
        {
            customerName = lastName;
        }
        // Break after processing the first customer
        break;
    }
}
// Insert the new delivery record linked to the order
response = insert into deliveries
[
    order=input.ID
    delivery_status="Pending"
    Customer_Name=customerName
    Added_User=zoho.loginuser
];
// Log the response for debugging
info response;
```

## C.2 Send Order Status Email

```
// Script ID : DS-02
// Location  : orders → On Edit → On Success
// Fetch the current order record using the ID from input
orderRecord = orders[ID == input.ID];
// Validate that the order record exists
if(orderRecord.isNull())
{
    info "Order record not found";
    return;
```

```
}
// Extract customer details from the order
// The 'customer' field is a multi-select lookup to
'customer_registrations'
// It returns a collection of IDs. We need to get the first ID.
customerCollection = orderRecord.customer;
if(customerCollection.isNull() || customerCollection.isEmpty())
{
    info "No customer associated with this order";
    return;
}
// Get the first customer ID from the collection
// Since 'customer' is a multi-select lookup, it returns a list of
IDs
firstCustomerID = customerCollection.get(0);
// Fetch the full customer registration details using the ID
customerDetails = customer_registrations[ID == firstCustomerID];
if(customerDetails.isNull())
{
    info "Customer details not found";
    return;
}
// Extract customer email and name
// 'full_name' is a NAME composite field
// 'email' is a direct field
customerEmail = customerDetails.email;
customerFirstName = customerDetails.full_name.first_name;
// Construct the email content
orderStatus = orderRecord.order_status;
orderID = orderRecord.order_id;
orderDate = orderRecord.order_date;
subject = "Your Order #" + orderID.toString() + " Status Update";
message = "<p>Dear " + customerFirstName + ",</p>";
message = message + "<p>We are writing to inform you that the
status of your order #" + orderID.toString() + " has been
updated.</p>";
message = message + "<p><strong>Current Status:</strong> " +
orderStatus + "</p>";
message = message + "<p><strong>Order Date:</strong> " +
orderDate.toString() + "</p>";
message = message + "<p>Thank you for shopping with us.</p>";
// Send the email
if(!customerEmail.isNull() && !customerEmail.isEmpty())
{
    sendmail
    [
        from :zoho.adminuserid
        to :customerEmail
        subject :subject
        message :message
        content type :HTML
    ]
    info "Email sent successfully to " + customerEmail;
}
else
{
    info "Customer email is missing, cannot send notification";
}
```

### C.3 Set Default Registration Values

```
// Script ID : DS-03
```

```
// Location  : customer_registrations → On Add → On Load
// Set registration_date to current date
input.registration_date = today;
// Set status to Active
input.status = "Active";
```

### C.4 Block Duplicate Customer

```
// Script ID : DS-04
// Location  : customer_registrations → On Add or Edit → On
Validate
// Check if email already exists
if(!input.email.isNull())
{
    existingCustomer = customer_registrations[email ==
input.email];
    if(existingCustomer.count() > 0)
    {
        alert "Customer already registered";
        cancel submit;
    }
}
// Check if phone number already exists
if(!input.phone_number.isNull())
{
    existingCustomer = customer_registrations[phone_number ==
input.phone_number];
    if(existingCustomer.count() > 0)
    {
        alert "Customer already registered";
        cancel submit;
    }
}
```

### C.5 Customer Login Validation

```
// Script ID : DS-05
// Location  : customer_login → On Add or Edit → On Validate
// Fetch customer registration record by email
registeredCustomer = customer_registrations[email == input.email];
// Check if email exists
if(registeredCustomer.ID.isNull())
{
    alert "Email not registered";
    cancel submit;
}
else
{
    // Check if account status is Inactive
    if(registeredCustomer.status == "Inactive")
    {
        alert "Account is inactive";
        cancel submit;
    }
    else
    {
        // Login successful - set a flag or update status if needed
        // Note: success message cannot be used in onValidate.
        // The form will proceed to submission if no cancel submit
is triggered.
        info "Login successful";
    }
```

```
}
```

### C.6 Block Duplicate Category Name

```
// Script ID : DS-06
// Location  : product_categories → On Add or Edit → On Validate
// Check if category_name already exists in the product_categories
form
existingCategory = product_categories[category_name ==
input.category_name];
if(existingCategory.count() > 0)
{
    alert "A category with this name already exists. Please use a
unique name.";
    cancel submit;
}
```

### C.7 Update Product Status on Save

```
// Script ID : DS-07
// Location  : products → On Add or Edit → On Success
if(input.stock_quantity <= 0)
{
    input.status = "Out of Stock";
}
else
{
    input.status = "Available";
}
```

### C.8 Validate Product Fields

```
// Script ID : DS-08
// Location  : products → On Add or Edit → On Validate
if(input.price < 0)
{
    alert(price,"Price cannot be negative");
    cancel submit;
}
else if(input.stock_quantity < 0)
{
    alert(stock_quantity,"Stock quantity cannot be negative");
    cancel submit;
}
else if(input.discount_percent > 100)
{
    alert(discount_percent,"Discount percentage cannot exceed
100");
    cancel submit;
}
```

### C.9 Validate Product Availability on Submit

```
// Script ID : DS-09
// Location  : order_booking → On Add or Edit → On Validate
// Fetch the product record based on the input product name
productRecord = products[product_name == input.product_name];
// Initialize validation flag
isValid = true;
errorMessage = "";
// Check if product exists
if(productRecord.ID.isNull())
{
    isValid = false;
```

```
    errorMessage = "Product not found";
}
else
{
    // Check if product status is Out of Stock
    if(productRecord.status == "Out of Stock")
    {
        isValid = false;
        errorMessage = "Product is out of stock";
    }
    // Check if requested quantity exceeds available stock
    else if(input.quantity > productRecord.stock_quantity)
    {
        isValid = false;
        errorMessage = "Quantity exceeds available stock
(Available: " + productRecord.stock_quantity + ")";
    }
}
// Block submission if validation fails
if(!isValid)
{
    alert errorMessage;
    cancel submit;
}
```

### C.10 Calculate Unit Price on Product Input

```
// Script ID : DS-10
// Location  : order_booking → On Add or Edit → On User Input of
product_name
// Fetch the product record matching the input product name
productRecord = products[product_name == input.product_name];
// Check if a matching product was found
if(productRecord.count() > 0)
{
    // Get the base price and discount percentage from the product
record
    basePrice = productRecord.price;
    discountPercent = productRecord.discount_percent;
    // Calculate the discount amount
    discountAmount = basePrice * discountPercent / 100;
    // Calculate the final unit price
    calculatedUnitPrice = basePrice - discountAmount;
    // Set the unit_price field in the current order_booking record
    input.unit_price = calculatedUnitPrice;
    // Optional: Update other fields if needed (e.g., product_name,
quantity)
    // input.product_name = productRecord.product_name;
}
else
{
    // Handle case where no matching product is found
    info "No product found with name: " + input.product_name;
    input.unit_price = 0;
}
```

### C.11 Set Booking Date to Today on Load

```
// Script ID : DS-11
// Location  : order_booking → On Add or Edit → On Load
input.booking_date = today;
```

### C.12 Create Delivery on Shipped

```
// Script ID : DS-12
// Location  : orders → On Edit → On Success
// Fetch the order record that was just submitted
orderRecord = orders[ID == input.ID];
// Check if order status is 'Shipped'
if(orderRecord.order_status == "Shipped")
{
    // Create a new delivery record with Shipped status
    // Note: orders form does not have customer_name field, using
customer lookup to get name
    customerRecord = customer_registrations[ID ==
orderRecord.customer.get(0)];
    // Construct full name from composite field if available,
otherwise use first_name
    if(customerRecord.full_name.first_name != null)
    {
        customerName = customerRecord.full_name.first_name;
        if(customerRecord.full_name.last_name != null)
        {
            customerName = customerName + " " +
customerRecord.full_name.last_name;
        }
    }
    else
    {
        customerName = "Customer";
    }
    deliveryResponse = insert into deliveries
    [
        Customer_Name=customerName
        order=orderRecord.ID
        delivery_status="Shipped"
        shipped_date=zoho.currentdate
        courier="Professional Couriers"
        tracking_number=""
        remarks="Order shipped successfully"
        Added_User=zoho.loginuser
    ];
    // Log the created delivery record ID
    info "Delivery record created with ID: " + deliveryResponse;
}
```

### C.13 Set Default Delivery Status

```
// Script ID : DS-13
// Location  : deliveries → On Add → On Load
// Set the delivery_status field to Pending when the form loads
input.delivery_status = "Pending";
```

### C.14 Validate Delivery Date

```
// Script ID : DS-14
// Location  : deliveries → On Add or Edit → On Validate
if(input.expected_delivery_date < input.shipped_date)
{
    alert(expected_delivery_date,"Expected delivery date cannot be
before shipped date");
    cancel submit;
}
```

### C.15 Delivery Status Reminder

```
// Script ID : DS-15
// Location  : Schedule on deliveries (1 day after
expected_delivery_date, 10:00)
// Check if delivery status is not 'Delivered'
if(input.delivery_status != "Delivered")
{
    // Fetch admin email address
    adminEmail = thisapp.permissions.adminEmailIds();
    // Construct email subject and message
    emailSubject = "Delivery Alert: Status is " +
input.delivery_status;
    emailMessage = "<p>Dear Admin,</p><p>The delivery with Tracking
Number: " + input.tracking_number + " is currently in status: " +
input.delivery_status + ".</p><p>Please take necessary
action.</p><p>Expected Delivery Date: " +
input.expected_delivery_date + "</p>";
    // Send email to admin
    sendmail
    [
        from :zoho.adminuserid
        to :adminEmail
        subject :emailSubject
        message :emailMessage
        content type :HTML
    ]
}
```

### C.16 Mark as Delivered Workflow

```
// Script ID : DS-16
// Location  : Custom action on orders (for each record)
// Update the order status to "Delivered"
input.order_status = "Delivered";
// Fetch the customer details from the order record
customerRecord = customer_registrations[ID == input.customer.ID];
// Prepare notification details
notificationTitle = "Order Delivered";
notificationMessage = "Your order with ID " + input.order_id + "
has been successfully delivered.";
// Send push notification to the customer
if(!customerRecord.isNull())
{
    notificationResponse =
zoho.pushNotification(customerRecord.email,notificationTitle,notifi
cationMessage,"orders",input.ID);
    info "Notification sent to " + customerRecord.email;
}
else
{
    info "Customer record not found for order " + input.order_id;
}
```

### C.17 add_sample_catalog

```
// Script ID : DS-17
// Location  : Standalone function
void add_sample_catalog()
{
    // Define list of category records
    categories = List();
```

```
categories.add({"category_name":"Electronics","description":"Electr
onic devices and accessories","status":"Active"});

categories.add({"category_name":"Clothing","description":"Apparel
and fashion items","status":"Active"});
    categories.add({"category_name":"Home &
Garden","description":"Home improvement and garden
supplies","status":"Active"});
    // Insert categories if they don't exist
    for each  category in categories
    {
        existingCategory = product_categories[category_name ==
category.get("category_name")];
        if(existingCategory.count() == 0)
        {
            insert into product_categories
            [
                category_name=category.get("category_name")
                description=category.get("description")
                status=category.get("status")
                Added_User=zoho.loginuser
            ]
        }
    }
    // Define list of product records
    products = List();

products.add({"product_name":"Smartphone","category_name":"Electron
ics","description":"Latest smartphone
model","price":599.99,"discount_percent":10,"stock_quantity":50,"st
atus":"Available"});
    products.add({"product_name":"T-
Shirt","category_name":"Clothing","description":"Cotton t-
shirt","price":19.99,"discount_percent":5,"stock_quantity":100,"sta
tus":"Available"});
    products.add({"product_name":"Garden
Hose","category_name":"Home & Garden","description":"50ft garden
hose","price":29.99,"discount_percent":0,"stock_quantity":30,"statu
s":"Available"});
    // Insert products if they don't exist
    for each  product in products
    {
        // Fetch category ID
        categoryRecord = product_categories[category_name ==
product.get("category_name")];
        categoryId = categoryRecord.ID;
        // Check if product exists
        existingProduct = products[product_name ==
product.get("product_name")];
        if(existingProduct.count() == 0)
        {
            insert into products
            [
                product_name=product.get("product_name")
                category={categoryId}
                description=product.get("description")
                price=product.get("price")
                discount_percent=product.get("discount_percent")
                stock_quantity=product.get("stock_quantity")
                status=product.get("status")
```

```
                Added_User=zoho.loginuser
            ]
        }
    }
}
```

# APPENDIX D – USER MANUAL

## D.1 Accessing the Application

- Open the application link https://creatorapp.zoho.in/dhanush.v.2024.cse_rajala366/e-commerce-management-system in a browser and sign in with the credentials for your role.
- On a phone or tablet, install the Zoho Creator app, sign in and open “E Commerce Management System”.

## D.2 Instructions for the Administrator

- Open Dashboard to see total orders, total revenue, delivery count and the status charts.
- Add categories under Product Categories, then products under Products; the status changes to Out of Stock when stock is zero.
- Register customers under Customer Registrations; duplicate e-mail or phone is rejected.
- Use Order Booking: type the product name, the unit price fills in automatically, then enter the quantity.
- Add orders and change order status; the customer receives an e-mail and a delivery record is created.
- Under Deliveries, set courier, tracking number and dates; use the Kanban view to follow status.
- Select an order and run Mark as Delivered when it reaches the customer.

## D.3 Instructions for the Customer (Portal User)

- Sign in to the portal and open Customer Registrations to register, if not already registered.
- Place an order through the available order form.
- Check the e-mail address given at registration for status updates.

# APPENDIX E – EVALUATION RUBRIC

*(For Examiner Use Only)*

| **Student Name** | Kishore.v | Register No. | 240701270 |
|---|---|---|---|
| **Project Title** | E-Commerce Management System | Date | 10 November 2026 |

Table E.1: Evaluation Rubric for the Individual Project

| S.No. | Criterion | Indicators | Max. Marks | Marks Awarded |
|---|---|---|---|---|
| 1 | Problem Definition & Requirement Analysis | Clarity of problem, measurable objectives, complete FR/NFR, use cases | 10 | |
| 2 | System Design | Architecture, ER/data model, form specifications, workflow design, permission matrix | 10 | |
| 3 | Forms, Reports & Pages | Variety of field types, lookups/subforms, report types, UI Builder dashboards | 15 | |
| 4 | Deluge Scripting & Workflows | Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows | 15 | |
| 5 | Integration, RBAC & AI | Zoho/external integrations, connections, roles & permissions, Zia/analytics | 10 | |
| 6 | Testing, Deployment, Security & Governance | Test coverage, environments, optimisation, security controls, audit | 10 | |
| 7 | Live Demonstration | End-to-end working of the application across roles; feature checklist verified | 15 | |
| 8 | Project Report | Adherence to template and format, clarity, figures, references | 5 | |
| 9 | Viva-Voce | Understanding of the platform, design decisions and low-code concepts | 10 | |
| | TOTAL | | 100 | |

Table E.2: Performance Level Descriptors

| Level | % of Max. Marks | Descriptor |
|---|---|---|
| Excellent | 90–100 | Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently. |
| Good | 70–89 | Feature implemented correctly with minor gaps; explanation adequate. |
| Satisfactory | 50–69 | Feature partially implemented or works only for the main path; limited explanation. |
| Needs Improvement | < 50 | Feature missing, non-functional or not understood by the student. |

Signature of Internal Examiner
Name: _____

Signature of External Examiner
Name: _____

# APPENDIX F – VIVA-VOCE RECORD SHEET

*(For Examiner Use Only)*

| | | | |
|---|---|---|---|
| **Student Name** | Kishore.v | Register No. | 240701270 |
| **Department / Section** | Computer Science and Engineering | Date of Viva | 10 November 2026 |
| **Application URL Verified** | Yes / No | Demo Duration | _____ min |

Table F.1: Viva-Voce Questions and Responses

| Q.No. | Question / Topic Asked | Unit | Response (E / G / S / NI) | Remarks |
|---|---|---|---|---|
| 1 | | | | |
| 2 | | | | |
| 3 | | | | |
| 4 | | | | |
| 5 | | | | |
| 6 | | | | |
| 7 | | | | |
| 8 | | | | |

E – Excellent  G – Good  S – Satisfactory  NI – Needs Improvement

Table F.2: Marks Summary

| Component | Max. Marks | Marks Awarded |
|---|---|---|
| Criteria 1–8 (Appendix E) | 90 | |
| Viva-Voce | 10 | |
| TOTAL | 100 | |

Examiners' Remarks:

_____

_____

**INTERNAL EXAMINER**
Name: _____

**EXTERNAL EXAMINER**
Name: _____